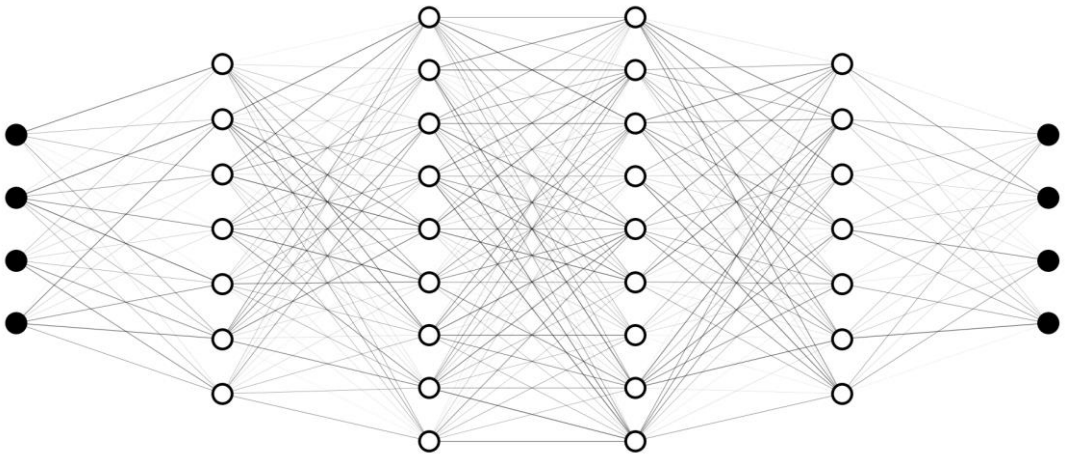
PROFESSIONAL DEVELOPMENT CURRICULUM

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

A Comprehensive Curriculum from Foundations to Advanced Practice

Basic • Intermediate • Advanced



LEVEL I **Foundations**	LEVEL II **Intermediate**	LEVEL III **Advanced**

November 2026

Document Information

Attribute	Detail
Document title	Artificial Intelligence and Machine Learning Engineering Curriculum
Subtitle	A Comprehensive Curriculum from Foundations to Advanced Practice
Edition	1.0
Date of issue	November 2026
Programme length	48 weeks full time or 96 weeks part time
Total study effort	Approximately 1,080 hours, comprising 720 guided and 360 independent hours
Structure	Three levels, twenty four modules and an integrated capstone project
Intended audience	Learners, instructors, training providers, employers and hiring managers
Review cycle	Every six months, with Level III content refreshed at each review

Version History

Version	Date	Summary of Changes
1.0	November 2026	First full edition covering all three levels, the capstone, assessment criteria and appendices.

This curriculum may be adopted, adapted and delivered by educational institutions, employers and independent learners. Where it is adapted, the module structure, learning outcomes and competency checkpoints should be retained so that the integrity of the progression from foundations to advanced practice is preserved.

Contents

Preface and Purpose of this Curriculum 5
How to Use this Document 5
Document Conventions 6
Programme Overview 7
Programme Aims 7
Target Learners and Entry Requirements 7
Programme Structure at a Glance 8
Programme Learning Outcomes 9
Learning and Teaching Strategy 10
Assessment Strategy 10
Recommended Technology Stack 11
Weekly Rhythm of Study 12
Level I: Foundations 13
Module 1: Python Programming for Machine Learning 13
Module 2: Linear Algebra for Machine Learning 16
Module 3: Calculus and Optimisation 18
Module 4: Probability and Statistics 20
Module 5: Data Acquisition, Wrangling and SQL 22
Module 6: Exploratory Data Analysis and Visualisation 24
Module 7: Software Engineering Foundations 26
Module 8: Supervised Learning Fundamentals 28
Level II: Intermediate Practice 31
Module 9: Unsupervised Learning and Ensembles 31
Module 10: Feature Engineering and Model Evaluation 33
Module 11: Neural Network Foundations with PyTorch 35
Module 12: Computer Vision with Deep Learning 37
Module 13: Natural Language Processing and Sequence Models 39
Module 14: Attention and the Transformer Architecture 41
Module 15: Time Series Forecasting and Recommender Systems 43
Module 16: MLOps I: From Notebook to Service 45
Level III: Advanced Engineering 48
Module 17: Large Language Models: Pretraining and Adaptation 48
Module 18: Retrieval Augmented Generation and AI Agents 50
Module 19: Deep Generative Models 53
Module 20: Reinforcement Learning 55
Module 21: Distributed Training and Efficient Inference 57
Module 22: MLOps II: Production Systems at Scale 59
Module 23: Responsible AI, Security and Governance 61
Module 24: ML System Design and Research Practice 63
Integrated Capstone Project 66

Purpose of the Capstone 66
Capstone Phases and Milestones 66
Indicative Capstone Themes 66
Capstone Assessment Scheme 67

Assessment Criteria and Grade Descriptors 68
Generic Grade Descriptors 68
Criteria for Technical Work 68
Academic Integrity and Use of AI Assistants 69

Competency Matrix and Career Pathways 70
Mapping of Modules to Programme Learning Outcomes 70
Professional Role Pathways 71
Indicative Competency Progression 71
Building a Professional Portfolio 72

Delivery Guidance for Instructors 73
Staffing and Support 73
Infrastructure Requirements 73
Inclusive Teaching Practice 73
Keeping the Curriculum Current 73

Appendices 74
Appendix A: Forty Eight Week Delivery Schedule 74
Appendix B: Core Reading List 75
Appendix C: Glossary of Key Terms 76
Appendix D: Preparatory Self Assessment 78
Closing Statement 79

Preface and Purpose of this Curriculum

Artificial intelligence and machine learning have moved, within a single decade, from specialist research topics into the operational core of almost every serious organisation. Banks rely on learned models to detect fraud in milliseconds, hospitals use computer vision to triage scans, retailers forecast demand at the level of a single product in a single store, and software teams now build entire products on top of large language models. This shift has created sustained demand for a particular kind of professional: the AI and machine learning engineer, who combines the mathematical understanding of a data scientist with the discipline, rigour and production instincts of a software engineer.

This curriculum has been written to produce exactly that professional. It is a complete, structured and assessable programme that takes a motivated learner from first principles to advanced practice. It begins with Python programming and the mathematics that underpins learning algorithms, progresses through classical machine learning, deep learning, computer vision and natural language processing, and culminates in large language models, retrieval augmented generation, generative modelling, reinforcement learning, distributed training, production machine learning operations and responsible AI governance.

The document is designed to serve several audiences at once. Independent learners can use it as a self study roadmap with clear milestones. Training providers, bootcamps and universities can adopt it as the backbone of a certificate or diploma programme. Employers can use it to structure graduate schemes, internal upskilling routes and apprenticeship standards. Hiring managers can use its competency checkpoints to calibrate interviews and technical assessments.

> *Guiding principle: every concept taught in this curriculum is paired with something the learner must build, measure and explain. Understanding is demonstrated through working systems, not through recitation.*

How to Use this Document

The curriculum is divided into three progressive levels. Level I establishes foundations, Level II develops intermediate practitioner capability, and Level III builds advanced engineering and research competence. Each level contains eight modules, and each module is designed to be completed in approximately two weeks of full time study or four weeks of part time study.

Every module follows an identical structure so that learners and instructors always know where to find what they need. Each module contains a summary table, an overview, measurable learning outcomes, a detailed syllabus broken into units, practical laboratories, an assessment scheme, recommended resources and a competency checkpoint. The checkpoint is the gate that a learner must pass before progressing.

Learners who already possess some of the required skills should not skip material on the basis of confidence alone. Instead, they should attempt the competency checkpoint for the relevant module. If they meet the standard, they may record the module as accredited through prior learning and move forward. If they do not, the gaps revealed by the checkpoint tell them precisely which units to study.

Document Conventions

Throughout this document the following conventions are used. Learning outcomes are written using active, measurable verbs drawn from a revised taxonomy of educational objectives, such as implement, derive, evaluate, design and justify. Guided hours refer to structured study including reading, lectures and laboratories. Independent hours refer to self directed practice, revision and project work. Weightings in assessment tables always total one hundred percent within a module.

Tool and library names are given in their commonly used form. Where a library has an official name that includes punctuation, it is written here in plain words, for example Scikit Learn for the popular Python machine learning library. Version numbers are deliberately avoided because the ecosystem moves quickly; learners should always use the current stable release recommended by each project.

Programme Overview

Programme Aims

The programme aims to develop engineers who can take an ambiguous business or scientific problem, frame it as a learning problem where appropriate, acquire and prepare the necessary data, select, train and rigorously evaluate suitable models, deploy those models into reliable production systems, and monitor and govern them responsibly over their full lifecycle. Specifically, the programme aims to:

1. Build a durable mathematical and statistical foundation so that learners understand why algorithms work, not merely how to call them.
2. Develop fluent, professional Python programming skills, including testing, version control, packaging and code review.
3. Provide broad and deep knowledge of classical machine learning, deep learning and modern foundation models.
4. Instil rigorous experimental habits, including sound validation design, reproducibility, ablation and honest reporting of uncertainty.
5. Equip learners to deploy, scale, monitor and maintain machine learning systems in real production environments.
6. Embed responsible AI practice, including fairness, privacy, security, transparency and regulatory compliance, into every stage of the lifecycle.
7. Cultivate the communication skills required to explain technical decisions to engineers, executives, regulators and the public.
8. Prepare learners for employment as machine learning engineers, applied scientists, AI engineers and MLOps engineers, or for progression to postgraduate research.

Target Learners and Entry Requirements

The programme is suitable for graduates in computing, engineering, mathematics, physics, statistics, economics and other quantitative disciplines. It is equally suitable for experienced software developers, data analysts and data scientists who wish to move into machine learning engineering, and for career changers from technical professions who can demonstrate numeracy and persistence.

Requirement	Expectation
Mathematics	Confidence with algebra, functions and basic graphs at upper secondary level. Prior exposure to calculus or matrices is helpful but is taught from first principles in Level I.

Requirement	Expectation
Programming	No prior programming is strictly required, but learners without any coding experience should complete a short preparatory course in Python before Module 1.
Hardware	A laptop with at least sixteen gigabytes of memory and a modern processor. A dedicated graphics processor is useful but not required, since cloud notebooks are used for intensive work.
Time commitment	Approximately fifteen hours per week for the full time route over forty eight weeks, or eight hours per week for the part time route over ninety six weeks.
Language	Ability to read technical documentation and write structured reports in English.

Programme Structure at a Glance

The programme comprises twenty four modules arranged across three levels, followed by an integrated capstone. The table below summarises the structure, the indicative scheduling for the full time route and the guided study hours for each module.

No.	Module Title	Level	Weeks	Hours
1	Python Programming for Machine Learning	I	1 to 2	30
2	Linear Algebra for Machine Learning	I	3 to 4	30
3	Calculus and Optimisation	I	5 to 6	30
4	Probability and Statistics	I	7 to 8	30
5	Data Acquisition, Wrangling and SQL	I	9 to 10	30
6	Exploratory Data Analysis and Visualisation	I	11 to 12	30
7	Software Engineering Foundations	I	13 to 14	30
8	Supervised Learning Fundamentals	I	15 to 16	30
9	Unsupervised Learning and Ensembles	II	17 to 18	30
10	Feature Engineering and Model Evaluation	II	19 to 20	30
11	Neural Network Foundations with PyTorch	II	21 to 22	30
12	Computer Vision with Deep Learning	II	23 to 24	30
13	Natural Language Processing and Sequence Models	II	25 to 26	30
14	Attention and the Transformer Architecture	II	27 to 28	30
15	Time Series Forecasting and Recommender Systems	II	29 to 30	30
16	MLOps I: From Notebook to Service	II	31 to 32	30

No.	Module Title	Level	Weeks	Hours
17	Large Language Models: Pretraining and Adaptation	III	33 to 34	30
18	Retrieval Augmented Generation and AI Agents	III	35 to 36	30
19	Deep Generative Models	III	37 to 38	30
20	Reinforcement Learning	III	39 to 40	30
21	Distributed Training and Efficient Inference	III	41 to 42	30
22	MLOps II: Production Systems at Scale	III	43 to 44	30
23	Responsible AI, Security and Governance	III	45 to 46	30
24	ML System Design and Research Practice	III	47 to 48	30

In addition to the guided hours shown above, learners should expect approximately fifteen independent hours per module, giving a total programme effort of roughly one thousand and eighty hours. The integrated capstone project, described in a later chapter, runs in parallel with Level III and draws on every preceding module.

Programme Learning Outcomes

On successful completion of the full programme, the learner will be able to demonstrate the following outcomes. Each outcome is mapped to the modules that develop it in the competency matrix contained in the appendices.

Code	Programme Learning Outcome
PLO1	Apply linear algebra, calculus, probability and statistics to analyse, derive and explain the behaviour of machine learning algorithms.
PLO2	Write clean, tested, documented and version controlled Python code that meets professional engineering standards.
PLO3	Acquire, clean, transform and validate data from heterogeneous sources, including relational databases, files and application programming interfaces.
PLO4	Select, implement, tune and critically evaluate classical machine learning algorithms for regression, classification, clustering and dimensionality reduction.
PLO5	Design, train, debug and optimise deep neural networks for vision, language, sequential and tabular problems.
PLO6	Build, adapt and evaluate applications based on large language models, including retrieval augmented generation and tool using agents.
PLO7	Deploy, scale, monitor and maintain machine learning systems in production using modern MLOps practices and cloud infrastructure.
PLO8	Identify and mitigate risks relating to fairness, privacy, security, safety and regulatory compliance throughout the AI lifecycle.

Code	Programme Learning Outcome
PLO9	Design end to end machine learning systems that balance accuracy, latency, cost, reliability and maintainability.
PLO10	Communicate technical work clearly through code, documentation, reports, presentations and stakeholder conversations.

Learning and Teaching Strategy

The programme follows a deliberate pedagogical model that can be summarised as concept, construction, critique. In the concept phase, learners meet new ideas through concise reading, worked derivations and short lectures. In the construction phase, they implement those ideas, frequently from scratch first and then using industrial libraries, so that the library ceases to be a black box. In the critique phase, they evaluate what they have built against baselines, examine failure cases and write short reflective analyses.

Several principles shape the teaching throughout:

- Implementation before abstraction. Learners implement linear regression, gradient descent, backpropagation and attention by hand before relying on frameworks.
- Spaced repetition and interleaving. Core ideas such as bias and variance, regularisation and validation design recur across modules in increasingly sophisticated forms.
- Authentic data. Wherever possible, laboratories use messy, real world datasets rather than curated toy examples, so that learners confront missing values, leakage, imbalance and drift.
- Engineering discipline from day one. Every laboratory submission is a version controlled repository with a readable README, a reproducible environment and automated tests.
- Feedback rich assessment. Formative quizzes, code reviews and peer critique precede every summative assessment.
- Professional communication. Every module includes at least one written or oral output aimed at a non specialist audience.

Assessment Strategy

Assessment is designed to measure capability rather than recall. Each module combines several components, typically a portfolio of laboratory work, a technical assignment or mini project, and either a written examination or a viva voce. Weightings vary by module and are listed in each module specification.

Assessment Type	Purpose and Description
Laboratory portfolio	A collection of completed laboratory notebooks and scripts, assessed for correctness, code quality, reproducibility and depth of analysis.

Assessment Type	Purpose and Description
Technical assignment	A structured problem requiring the learner to implement and evaluate a solution, accompanied by a short technical report.
Mini project	An open ended applied problem in which the learner defines scope, selects methods and justifies decisions.
Written examination	A timed assessment of conceptual understanding, derivations and the ability to reason about unfamiliar scenarios.
Viva voce	An oral examination in which the learner explains and defends their work, demonstrating genuine understanding.
Design review	A presentation of a system design to a panel, followed by questioning on trade offs and risks.

The pass mark for every module is fifty percent, with a minimum of forty percent required in each individual component. Merit is awarded for an overall average of sixty percent and distinction for an overall average of seventy percent, subject to a distinction level capstone. Learners who do not pass a component may resubmit once, with the resubmitted mark capped at the pass threshold.

Recommended Technology Stack

Learners are expected to become fluent in a carefully chosen stack of industry standard tools. The stack is introduced progressively so that no learner faces too many new tools at once.

Category	Tools and Platforms
Language and environment	Python, Jupyter notebooks, Visual Studio Code, Conda or uv for environment management
Numerical computing	NumPy, SciPy, Pandas, Polars
Visualisation	Matplotlib, Seaborn, Plotly
Classical machine learning	Scikit Learn, XGBoost, LightGBM, CatBoost, Statsmodels
Deep learning	PyTorch, PyTorch Lightning, TorchVision, Hugging Face Transformers, Datasets and PEFT
Large language models	Hugging Face Hub, vLLM, LangChain or LlamaIndex, vector databases such as FAISS, Qdrant or pgvector
Data and storage	PostgreSQL, SQLite, DuckDB, Parquet, object storage such as Amazon S3 or Azure Blob Storage
Engineering	Git, GitHub, pytest, Ruff, Black, mypy, pre commit hooks
MLOps	MLflow, Weights and Biases, DVC, Docker, FastAPI, GitHub Actions, Kubernetes, Prometheus, Grafana
Cloud	Amazon Web Services, Microsoft Azure or Google Cloud Platform, with emphasis on one provider in depth

Weekly Rhythm of Study

For the full time route, a typical week contains around fifteen guided hours and a similar quantity of independent work. The suggested rhythm below balances new learning, practice and consolidation.

Day	Activity
Monday	Reading and lectures introducing the week's concepts, with a short formative quiz.
Tuesday	Worked derivations and from scratch implementations in guided laboratories.
Wednesday	Applied laboratory using industrial libraries on a realistic dataset.
Thursday	Assignment or mini project work, with a scheduled code review clinic.
Friday	Consolidation, peer critique, reflective log entry and preparation for the following week.

The reflective log deserves special emphasis. Each week the learner records what they learned, what confused them, what they built and what they would do differently. Over forty eight weeks this log becomes a powerful record of growth and an excellent source of material for interviews and portfolio narratives.

Level I: Foundations

Level I establishes the bedrock on which every later capability rests. Many practitioners who struggle with advanced topics such as transformer optimisation or reinforcement learning do so not because those topics are inherently impenetrable, but because their grasp of vectors, gradients, probability distributions or clean code is fragile. Level I is therefore deliberately rigorous. It refuses to treat mathematics as an optional appendix and refuses to treat programming as mere scripting.

The level comprises eight modules delivered over sixteen weeks on the full time route. The first four modules build programming and mathematical fluency. The next three develop the practical skills of data handling, exploratory analysis and professional software engineering. The final module brings everything together in a first serious encounter with supervised machine learning, implemented both from scratch and with industrial libraries.

> *Level I exit standard: the learner can implement and evaluate a regularised linear or logistic regression model from scratch in NumPy, explain every line mathematically, reproduce the result in Scikit Learn, and deliver the work as a tested, version controlled repository.*

Module 1: Python Programming for Machine Learning

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 1 to 2	Entry requirements only	30 guided and 15 independent

Module Overview

Python is the lingua franca of machine learning. This module develops fluent, idiomatic and efficient Python programming with a specific focus on the patterns used in data science and machine learning codebases. Learners progress from core syntax to object oriented design, functional idioms, vectorised numerical computing with NumPy and tabular manipulation with Pandas. The emphasis throughout is on writing code that is correct, readable and fast enough, and on understanding why vectorised code dramatically outperforms naive loops.

Learning Outcomes

By the end of this module the learner will be able to:

1. Write idiomatic Python using appropriate built in data structures, comprehensions, generators and context managers.
2. Design small programs using functions, modules, classes, inheritance, composition and data classes.

3. Apply type hints and docstrings to produce self documenting code that passes static analysis.
4. Manipulate multidimensional arrays in NumPy using broadcasting, fancy indexing and vectorised operations.
5. Load, clean, reshape, join and aggregate tabular data using Pandas.
6. Profile Python code and justify performance improvements using empirical measurements.
7. Manage reproducible environments and dependencies for a data science project.

Syllabus

Unit 1.1 Core language and data structures. The Python execution model, dynamic typing and object identity; integers, floats and the limits of floating point precision; strings and formatted string literals; lists, tuples, dictionaries and sets, including their time complexity for common operations; mutability and its consequences for function arguments.

- Control flow, iteration protocols and the enumerate and zip functions.
- List, dictionary and set comprehensions, and when they harm readability.
- Exceptions, custom exception classes and defensive programming.

Unit 1.2 Functions and functional idioms. Positional, keyword, default and variadic arguments; scope and closures; first class functions; lambda expressions; map, filter and reduce; decorators for logging, timing and caching; generators and lazy evaluation for processing datasets larger than memory.

Unit 1.3 Object oriented programming. Classes, instances, attributes and methods; special methods for representation, comparison and iteration; inheritance versus composition; abstract base classes; data classes and named tuples; designing simple estimator classes with fit and predict methods that mirror the Scikit Learn interface.

Unit 1.4 Typing, documentation and style. Type hints including generics, optional and union types; static checking with mypy; docstring conventions; the PEP 8 style guide; automated formatting and linting with Black and Ruff.

Unit 1.5 Numerical computing with NumPy. The ndarray, data types and memory layout; creation routines; slicing, boolean masking and fancy indexing; broadcasting rules explained visually; universal functions; aggregation along axes; random number generation with seeded generators; linear algebra routines; writing loop free implementations of distance matrices and softmax.

Unit 1.6 Tabular data with Pandas. Series and DataFrame objects; reading CSV, Excel, JSON and Parquet files; selection with loc and iloc; handling missing values; group by, aggregation and transformation; merge, join and concatenation; pivoting and melting; working with dates and categorical types; method chaining for readable pipelines; an introduction to Polars as a high performance alternative.

Unit 1.7 Performance and environments. Timing with timeit; profiling with cProfile and line profilers; algorithmic complexity in practice; vectorisation versus loops; memory profiling; creating isolated environments with Conda or uv; pinning dependencies and lock files.

Practical Laboratories

- **Lab 1A: Python kata.** Twenty graded exercises covering data structures, comprehensions, closures and generators, with automated test feedback.
- **Lab 1B: Building a mini estimator.** Implement a nearest centroid classifier as a Python class with fit, predict and score methods and full type hints.
- **Lab 1C: Vectorisation challenge.** Rewrite loop based implementations of pairwise Euclidean distance, softmax and moving averages in NumPy and report speedups.
- **Lab 1D: Pandas data cleaning.** Clean and analyse a public transport punctuality dataset, producing tidy summary tables by route and month.

Assessment

Component	Weight	Description
Laboratory portfolio	40%	Completed labs 1A to 1D, assessed for correctness, style and test coverage.
Coding assignment	40%	Build a typed, tested data processing package that ingests raw files and produces analytical summaries.
Timed practical test	20%	A ninety minute live coding test on NumPy and Pandas manipulation.

Recommended Resources

- Wes McKinney, Python for Data Analysis, O'Reilly.
- Luciano Ramalho, Fluent Python, O'Reilly.
- Brett Slatkin, Effective Python, Addison Wesley.
- The official NumPy and Pandas user guides.

Competency Checkpoint

The learner is ready to progress when, given an unfamiliar CSV dataset and a written specification, they can produce within two hours a clean, typed and tested Python module that loads, validates, transforms and summarises the data using vectorised operations, without consulting step by step tutorials.

Module 2: Linear Algebra for Machine Learning

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 3 to 4	Module 1	30 guided and 15 independent

Module Overview

Linear algebra is the language in which machine learning is written. Data points are vectors, datasets are matrices, model parameters are tensors, and almost every computation in a neural network is a matrix multiplication. This module develops both geometric intuition and computational fluency. Every concept is introduced visually, formalised mathematically and implemented in NumPy, so that learners can move confidently between the picture, the equation and the code.

Learning Outcomes

By the end of this module the learner will be able to:

1. Represent data, transformations and model parameters using vectors, matrices and tensors.
2. Interpret matrix multiplication geometrically as composition of linear transformations.
3. Compute and interpret norms, inner products, projections and orthogonality.
4. Solve linear systems and explain rank, invertibility, null spaces and conditioning.
5. Compute and interpret eigenvalues, eigenvectors and the singular value decomposition.
6. Derive principal component analysis from first principles and implement it in NumPy.
7. Relate linear algebra concepts directly to regression, embeddings and neural network layers.

Syllabus

Unit 2.1 Vectors and vector spaces. Vectors as arrows and as lists of numbers; addition and scalar multiplication; linear combinations, span and linear independence; basis and dimension; subspaces; the interpretation of a dataset row as a point in feature space.

Unit 2.2 Norms, inner products and geometry. The L1, L2 and infinity norms and their role in regularisation; the dot product and cosine similarity; angles and orthogonality; orthogonal projection onto a line and onto a subspace; the Gram Schmidt process; distance metrics used in nearest neighbour methods and vector search.

Unit 2.3 Matrices as linear transformations. Matrix vector and matrix matrix multiplication; rotation, scaling, shear and projection matrices; composition of transformations; transpose, identity and inverse; special matrices including diagonal, symmetric, orthogonal and triangular; the computational cost of matrix operations.

Unit 2.4 Linear systems. Gaussian elimination and row echelon form; rank and the four fundamental subspaces; existence and uniqueness of solutions; overdetermined systems and the least squares solution through the normal equations; condition number and numerical stability; LU and QR decompositions.

Unit 2.5 Eigendecomposition. Eigenvalues and eigenvectors and their geometric meaning; characteristic polynomial; diagonalisation; symmetric matrices and the spectral theorem; positive definite and semidefinite matrices and their connection to covariance and convexity; power iteration.

Unit 2.6 Singular value decomposition. Definition and geometric interpretation; relationship to eigendecomposition; low rank approximation and the Eckart Young theorem; the pseudoinverse; applications in image compression, latent semantic analysis and recommender systems.

Unit 2.7 Principal component analysis and tensors. Derivation of principal component analysis as variance maximisation and as reconstruction error minimisation; explained variance; whitening; an introduction to tensors, tensor shapes and the batched operations used throughout deep learning frameworks.

Practical Laboratories

- **Lab 2A: Transformations visualised.** Apply rotation, scaling and shear matrices to a two dimensional shape and animate the results.
- **Lab 2B: Least squares from scratch.** Fit a linear regression using the normal equations, QR decomposition and the pseudoinverse, and compare numerical stability.
- **Lab 2C: Image compression with SVD.** Compress greyscale images at varying ranks and plot reconstruction error against storage.
- **Lab 2D: PCA from first principles.** Implement principal component analysis using eigendecomposition and SVD, validate against Scikit Learn and visualise a high dimensional dataset in two dimensions.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 2A to 2D with mathematical commentary accompanying each implementation.
Problem sheet	25%	Hand derivations and proofs covering projections, rank, eigenvalues and SVD.
Written examination	40%	A two hour examination combining computation, derivation and conceptual reasoning.

Recommended Resources

- Marc Peter Deisenroth, A. Aldo Faisal and Cheng Soon Ong, Mathematics for Machine Learning, Cambridge University Press.

- Gilbert Strang, Introduction to Linear Algebra, Wellesley Cambridge Press.
- The Essence of Linear Algebra video series by 3Blue1Brown.

Competency Checkpoint

The learner is ready to progress when they can explain, using a sketch and an equation, why the first principal component is the leading eigenvector of the covariance matrix, implement the computation in fewer than twenty lines of NumPy, and explain how the singular value decomposition generalises the result.

Module 3: Calculus and Optimisation

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 5 to 6	Modules 1 and 2	30 guided and 15 independent

Module Overview

Learning, in the machine learning sense, is optimisation: the search for parameters that minimise a loss function. This module equips learners with the differential calculus required to reason about that search and with the optimisation algorithms used to perform it. The module culminates in a from scratch implementation of automatic differentiation, which demystifies the engine at the heart of every modern deep learning framework.

Learning Outcomes

By the end of this module the learner will be able to:

1. Compute derivatives, partial derivatives, gradients, Jacobians and Hessians of functions used in machine learning.
2. Apply the chain rule to composite functions and represent computations as directed acyclic graphs.
3. Explain the distinction between forward mode and reverse mode automatic differentiation.
4. Implement gradient descent, stochastic gradient descent, momentum and adaptive optimisers.
5. Analyse convexity, local minima, saddle points and the effect of learning rate on convergence.
6. Formulate constrained optimisation problems using Lagrange multipliers.
7. Verify analytic gradients numerically using finite difference approximations.

Syllabus

Unit 3.1 Single variable calculus refresher. Limits and continuity; the derivative as a rate of change and as the slope of a tangent; rules of differentiation; derivatives of exponential, logarithmic and sigmoid functions; Taylor series and linear and quadratic approximations.

Unit 3.2 Multivariable calculus. Partial derivatives; the gradient as the direction of steepest ascent; directional derivatives; Jacobian matrices for vector valued functions; the Hessian and curvature; matrix calculus identities used in deriving least squares and logistic regression.

Unit 3.3 The chain rule and computational graphs. The multivariable chain rule; representing expressions as computational graphs; forward and backward passes; the backpropagation algorithm derived as reverse mode differentiation; computational and memory cost trade offs.

Unit 3.4 Convexity and optimality. Convex sets and convex functions; first and second order conditions; why convex problems have no spurious local minima; the loss landscapes of neural networks, saddle points and plateaus.

Unit 3.5 Gradient based optimisation. Batch gradient descent; stochastic and mini batch gradient descent; learning rate selection and schedules; momentum and Nesterov acceleration; AdaGrad, RMSProp, Adam and AdamW; gradient clipping; an introduction to second order methods including Newton's method and L BFGS.

Unit 3.6 Constrained optimisation. Lagrange multipliers; the Karush Kuhn Tucker conditions; duality; application to the derivation of support vector machines and to regularisation viewed as a constraint.

Unit 3.7 Integration for probability. Definite integrals as areas and accumulated quantities; integration of probability density functions; expectation as an integral; Monte Carlo estimation of integrals as a bridge to Module 4.

Practical Laboratories

- **Lab 3A: Gradient checking.** Implement analytic and finite difference gradients for several loss functions and quantify the discrepancy.
- **Lab 3B: Optimiser race.** Implement gradient descent, momentum, RMSProp and Adam and visualise their trajectories on the Rosenbrock and Beale functions.
- **Lab 3C: Micro autograd.** Build a scalar valued automatic differentiation engine supporting addition, multiplication, power and nonlinear functions, then train a tiny neural network with it.
- **Lab 3D: Learning rate sensitivity.** Run a systematic study of learning rate and batch size on a logistic regression problem and present the results.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 3A to 3D including visualisations and written interpretation.
Autograd project	35%	A documented, tested automatic differentiation engine with a worked training example.

Component	Weight	Description
Written examination	30%	Derivations of gradients, chain rule applications and optimisation analysis.

Recommended Resources

- Deisenroth, Faisal and Ong, Mathematics for Machine Learning, chapters on vector calculus and continuous optimisation.
- Stephen Boyd and Lieven Vandenberghe, Convex Optimization, Cambridge University Press.
- Andrej Karpathy, the micrograd lecture and repository.

Competency Checkpoint

The learner is ready to progress when they can derive by hand the gradient of the binary cross entropy loss with respect to the weights of a logistic regression model, implement it, verify it numerically, and explain how the same computation would be performed automatically by reverse mode differentiation.

Module 4: Probability and Statistics

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 7 to 8	Modules 1 to 3	30 guided and 15 independent

Module Overview

Machine learning is fundamentally about reasoning under uncertainty. Models are fitted to samples, predictions carry error, and every evaluation metric is itself a random variable. This module develops probabilistic thinking and statistical inference with a sustained focus on their use in machine learning: likelihood based learning, Bayesian reasoning, uncertainty quantification and the design and interpretation of experiments.

Learning Outcomes

By the end of this module the learner will be able to:

1. Apply the axioms of probability, conditional probability, independence and Bayes' theorem.
2. Characterise and simulate common discrete and continuous probability distributions.
3. Compute and interpret expectation, variance, covariance and correlation.
4. Derive maximum likelihood and maximum a posteriori estimators for standard models.
5. Construct confidence intervals and conduct hypothesis tests, interpreting results correctly.
6. Design and analyse A/B experiments, including power and sample size calculations.
7. Use bootstrap resampling to quantify uncertainty in model performance metrics.

Syllabus

Unit 4.1 Foundations of probability. Sample spaces, events and axioms; conditional probability; the law of total probability; independence; Bayes' theorem with worked examples in medical testing and spam filtering; common fallacies including base rate neglect.

Unit 4.2 Random variables and distributions. Discrete distributions including Bernoulli, binomial, categorical and Poisson; continuous distributions including uniform, exponential, Gaussian, multivariate Gaussian and Beta; cumulative distribution functions; joint, marginal and conditional distributions.

Unit 4.3 Moments and limit theorems. Expectation and its linearity; variance and standard deviation; covariance and correlation matrices; the law of large numbers; the central limit theorem demonstrated through simulation; why averages of noisy measurements stabilise.

Unit 4.4 Estimation. Estimators, bias, variance and mean squared error; maximum likelihood estimation for Bernoulli, Gaussian and Poisson models; the equivalence between least squares and Gaussian likelihood and between cross entropy and Bernoulli likelihood; maximum a posteriori estimation and its relationship to regularisation.

Unit 4.5 Bayesian inference. Priors, likelihoods and posteriors; conjugate priors; posterior predictive distributions; an introduction to Markov chain Monte Carlo sampling; when Bayesian methods are worth their additional cost.

Unit 4.6 Statistical inference and testing. Sampling distributions; confidence intervals; hypothesis tests including Student's t test, chi squared test and permutation tests; p values and their frequent misinterpretation; multiple comparisons and the false discovery rate; effect sizes.

Unit 4.7 Experimentation and uncertainty. Designing A/B tests; statistical power and sample size; sequential testing pitfalls; bootstrap confidence intervals for accuracy, area under the curve and other metrics; information theory essentials including entropy, cross entropy and Kullback Leibler divergence.

Practical Laboratories

- **Lab 4A: Simulation studio.** Simulate the central limit theorem, the birthday problem and the Monty Hall problem, comparing simulated and analytic answers.
- **Lab 4B: Maximum likelihood.** Fit Gaussian and Poisson models to real data by maximum likelihood, both analytically and with numerical optimisation.
- **Lab 4C: Bayesian coin and click through rate.** Implement Beta Binomial updating for an advertising click through problem and visualise posterior evolution.
- **Lab 4D: A/B test analysis.** Analyse a simulated product experiment, calculate power, test significance and write a decision memo for a product manager.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 4A to 4D, with clear statistical interpretation.
Applied statistics report	30%	An analysis of a real dataset using estimation, testing and bootstrap methods, written for a non specialist reader.
Written examination	35%	Probability calculations, estimator derivations and interpretation of statistical output.

Recommended Resources

- Joseph Blitzstein and Jessica Hwang, Introduction to Probability, CRC Press.
- Peter Bruce, Andrew Bruce and Peter Gedeck, Practical Statistics for Data Scientists, O'Reilly.
- Kevin Murphy, Probabilistic Machine Learning: An Introduction, MIT Press.

Competency Checkpoint

The learner is ready to progress when they can explain to a non technical stakeholder why a model with ninety two percent accuracy on four hundred test examples might not be meaningfully better than one with ninety percent, and support the argument with a bootstrap confidence interval computed in code.

Module 5: Data Acquisition, Wrangling and SQL

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 9 to 10	Modules 1 and 4	30 guided and 15 independent

Module Overview

Practitioners routinely report that the majority of effort in a machine learning project is spent on data rather than models. This module develops the skills needed to acquire data from databases, files, application programming interfaces and the web, and to transform it into reliable, well documented, analysis ready datasets. Particular emphasis is placed on SQL, which remains the most important data language in industry, and on data quality and validation.

Learning Outcomes

By the end of this module the learner will be able to:

1. Write correct and efficient SQL queries, including joins, aggregations, subqueries, common table expressions and window functions.
2. Design simple relational schemas and explain normalisation and indexing.

3. Acquire data from REST application programming interfaces, files and web pages responsibly and lawfully.
4. Clean data by handling missing values, duplicates, outliers, inconsistent formats and type errors.
5. Implement automated data validation checks and data quality reports.
6. Build reproducible extract, transform and load pipelines using Python and SQL.
7. Explain the principles of data protection that apply to personal data in machine learning projects.

Syllabus

Unit 5.1 Relational databases and SQL fundamentals. The relational model; tables, keys and relationships; SELECT, WHERE, ORDER BY and LIMIT; aggregate functions with GROUP BY and HAVING; inner, left, right and full outer joins; set operations; NULL semantics and three valued logic.

Unit 5.2 Advanced SQL for analytics. Subqueries and correlated subqueries; common table expressions including recursive queries; window functions for ranking, running totals, lags and moving averages; date arithmetic; query plans, indexes and performance tuning; analytical engines such as DuckDB.

Unit 5.3 Schema design and data modelling. Normal forms; star and snowflake schemas for analytics; slowly changing dimensions; the distinction between transactional and analytical workloads; data warehouses and lakehouses at a conceptual level.

Unit 5.4 Acquiring data. Working with CSV, JSON, Parquet and Excel files; consuming REST application programming interfaces with authentication, pagination and rate limiting; responsible web scraping, including robots directives and terms of service; working with public open data portals.

Unit 5.5 Cleaning and transformation. Systematic approaches to missing data, including understanding missingness mechanisms; deduplication and record linkage; outlier detection; string normalisation and regular expressions; parsing dates and time zones; reshaping between wide and long formats.

Unit 5.6 Data quality and validation. Dimensions of data quality including completeness, accuracy, consistency, timeliness and uniqueness; schema validation with Pandera or Great Expectations; automated data quality reports; data contracts between producers and consumers.

Unit 5.7 Data governance and protection. Principles of the UK General Data Protection Regulation and the Data Protection Act 2018 as they affect machine learning; lawful bases for processing; data minimisation; pseudonymisation and anonymisation; data lineage and documentation through datasheets.

Practical Laboratories

- **Lab 5A: SQL gauntlet.** Forty progressively harder SQL problems on a retail database, culminating in cohort retention analysis with window functions.
- **Lab 5B: API harvester.** Build a resilient client for a public weather or transport API with retries, pagination and caching.

- **Lab 5C: Messy data rescue.** Clean a deliberately corrupted healthcare appointments dataset and document every transformation.
- **Lab 5D: Validated pipeline.** Build an extract, transform and load pipeline with schema validation that fails loudly on bad input.

Assessment

Component	Weight	Description
Laboratory portfolio	30%	Labs 5A to 5D, including query files and pipeline code.
SQL examination	30%	A timed practical examination of analytical SQL.
Data pipeline project	40%	An end to end pipeline combining at least two sources into a validated analytical dataset, with a datasheet.

Recommended Resources

- Alan Beaulieu, Learning SQL, O'Reilly.
- Martin Kleppmann, Designing Data Intensive Applications, O'Reilly, early chapters on data models.
- Information Commissioner's Office guidance on AI and data protection.

Competency Checkpoint

The learner is ready to progress when they can take two raw sources with conflicting identifiers and formats, produce a single validated table with documented lineage, and answer a multi part business question using window functions in a single well structured SQL query.

Module 6: Exploratory Data Analysis and Visualisation

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 11 to 12	Modules 1, 4 and 5	30 guided and 15 independent

Module Overview

Before any model is trained, the engineer must understand the data: its structure, its distributions, its relationships, its anomalies and its biases. This module teaches a disciplined approach to exploratory data analysis and the principles of clear, honest and persuasive visualisation. Learners will discover that many modelling failures, including leakage and spurious correlations, can be detected early by careful exploration.

Learning Outcomes

By the end of this module the learner will be able to:

1. Conduct a structured exploratory analysis that profiles variables, relationships and data quality issues.
2. Select appropriate chart types for distributions, comparisons, compositions, relationships and trends.
3. Produce publication quality static visualisations and interactive dashboards.
4. Identify potential sources of bias, leakage and confounding during exploration.
5. Apply dimensionality reduction techniques to visualise high dimensional data.
6. Communicate analytical findings through a coherent narrative tailored to its audience.

Syllabus

Unit 6.1 A framework for exploration. Formulating questions; univariate, bivariate and multivariate analysis; the role of summary statistics and their limitations, illustrated by Anscombe's quartet; automated profiling tools and their appropriate use.

Unit 6.2 Principles of visual perception. Preattentive attributes; the hierarchy of visual encodings; the data ink ratio; colour theory for categorical, sequential and diverging data; accessibility and colour blindness; common misleading practices such as truncated axes.

Unit 6.3 Visualisation with Python. Matplotlib architecture of figures and axes; Seaborn for statistical graphics; Plotly for interactive charts; small multiples and faceting; annotation and storytelling techniques.

Unit 6.4 Analysing distributions and relationships. Histograms, kernel density estimates, box plots and violin plots; scatter plots and hexbin plots; correlation matrices and their pitfalls; Simpson's paradox; categorical associations; detecting heteroscedasticity and nonlinearity.

Unit 6.5 Temporal and geospatial exploration. Time series decomposition into trend, seasonality and residual; rolling statistics; calendar effects; an introduction to geospatial visualisation with choropleth maps.

Unit 6.6 High dimensional visualisation. Principal component projections revisited; t distributed stochastic neighbour embedding and uniform manifold approximation and projection; their hyperparameters and the dangers of overinterpretation.

Unit 6.7 Data storytelling and dashboards. Structuring an analytical narrative; executive summaries; building simple dashboards with Streamlit; tailoring communication to technical and non technical audiences.

Practical Laboratories

- **Lab 6A: Chart critique.** Redesign five poor visualisations taken from public reports and justify every design change.
- **Lab 6B: Structured EDA.** Perform a complete exploratory analysis on a housing dataset and identify at least three modelling risks.
- **Lab 6C: Embedding explorer.** Visualise a high dimensional dataset with PCA, tSNE and UMAP and compare the stories each tells.
- **Lab 6D: Dashboard build.** Build an interactive Streamlit dashboard for a public health or transport dataset.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 6A to 6D with design rationale.
EDA report	40%	A full exploratory analysis report on an unseen dataset, including modelling recommendations.
Presentation	25%	A ten minute data story delivered to a mixed technical and non technical panel.

Recommended Resources

- Cole Nussbaumer Knaflic, Storytelling with Data, Wiley.
- Claus Wilke, Fundamentals of Data Visualization, O'Reilly.
- Edward Tufte, The Visual Display of Quantitative Information, Graphics Press.

Competency Checkpoint

The learner is ready to progress when, given a new dataset, they can within one working day produce a concise EDA report that identifies the target distribution, the most informative features, data quality issues, likely leakage risks and a recommended modelling approach.

Module 7: Software Engineering Foundations

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 13 to 14	Module 1	30 guided and 15 independent

Module Overview

The difference between a data scientist who produces notebooks and an engineer who produces systems lies largely in software engineering discipline. This module introduces the tools and habits of

professional software development: the command line, version control, testing, packaging, code review, documentation and continuous integration. These habits are applied in every subsequent module, and their absence is the most common reason that promising machine learning prototypes never reach production.

Learning Outcomes

By the end of this module the learner will be able to:

1. Work productively in a Linux shell, including file manipulation, process management and scripting.
2. Use Git for branching, merging, rebasing and resolving conflicts within a collaborative workflow.
3. Write unit, integration and property based tests using pytest, including fixtures and mocking.
4. Structure Python projects as installable packages with clear module boundaries.
5. Configure continuous integration pipelines that lint, type check and test code automatically.
6. Apply design principles such as separation of concerns, dependency injection and configuration management.
7. Participate constructively in code review as both author and reviewer.

Syllabus

Unit 7.1 The command line. Shell navigation and file operations; pipes and redirection; text processing with grep, sed and awk; environment variables; permissions; processes and signals; secure shell access to remote machines; writing simple shell scripts.

Unit 7.2 Version control with Git. The Git object model; commits, branches and tags; merging versus rebasing; resolving conflicts; remote repositories; pull requests; branching strategies such as trunk based development; writing meaningful commit messages; handling large files and secrets safely.

Unit 7.3 Testing. The testing pyramid; unit tests with pytest; fixtures and parametrisation; mocking external services; property based testing with Hypothesis; testing numerical code with tolerances; testing data pipelines and machine learning code, including deterministic seeding.

Unit 7.4 Project structure and packaging. The source layout; pyproject configuration; building and installing packages; entry points and command line interfaces with Typer or Click; configuration with environment variables and structured configuration files.

Unit 7.5 Code quality and design. Linting and formatting; static typing at scale; pre commit hooks; SOLID principles applied pragmatically; refactoring notebooks into modules; logging rather than printing; error handling strategies.

Unit 7.6 Continuous integration. GitHub Actions workflows; matrix builds; caching dependencies; test coverage reporting; branch protection rules; semantic versioning and release notes.

Unit 7.7 Collaboration and documentation. Writing excellent README files; docstrings and generated documentation with MkDocs; architecture decision records; the etiquette and value of code review; issue tracking and agile practices in machine learning teams.

Practical Laboratories

- **Lab 7A: Shell scavenger hunt.** Solve a series of file and log analysis puzzles using only shell tools.
- **Lab 7B: Git collaboration drill.** Work in pairs to create, review and merge feature branches, deliberately resolving conflicts.
- **Lab 7C: Notebook to package.** Refactor an exploratory notebook into a tested, typed and installable package with a command line interface.
- **Lab 7D: CI pipeline.** Configure a continuous integration workflow that blocks merges on failing tests, lint errors or type errors.

Assessment

Component	Weight	Description
Laboratory portfolio	30%	Labs 7A to 7D evidenced through repository history.
Engineering project	50%	A packaged, tested, documented library with continuous integration and at least eighty percent test coverage.
Code review exercise	20%	A written review of a peer's pull request, assessed for accuracy, tact and usefulness.

Recommended Resources

- David Thomas and Andrew Hunt, The Pragmatic Programmer, Addison Wesley.
- Scott Chacon and Ben Straub, Pro Git, Apress, freely available online.
- Brian Okken, Python Testing with pytest, Pragmatic Bookshelf.

Competency Checkpoint

The learner is ready to progress when every repository they create contains a clear README, a reproducible environment specification, automated tests and a passing continuous integration workflow, without prompting.

Module 8: Supervised Learning Fundamentals

Level	Schedule	Prerequisites	Study Hours
Level I Foundations	Weeks 15 to 16	Modules 1 to 7	30 guided and 15 independent

Module Overview

This capstone module of Level I introduces supervised learning, the most widely deployed form of machine learning. Learners implement core algorithms from scratch, understand them through the lens of loss functions, optimisation and probability, and then reproduce and extend them using Scikit Learn. The module establishes the vocabulary and mental models that underpin the remainder of the programme, most importantly the bias and variance trade off, overfitting, regularisation and sound validation.

Learning Outcomes

By the end of this module the learner will be able to:

1. Frame real world problems as regression or classification tasks with appropriate loss functions and metrics.
2. Implement linear regression, logistic regression and k nearest neighbours from scratch.
3. Explain and diagnose underfitting and overfitting using learning curves and the bias and variance decomposition.
4. Apply L1, L2 and elastic net regularisation and interpret their effects on model coefficients.
5. Train and interpret decision trees, support vector machines and naive Bayes classifiers.
6. Design train, validation and test splits and apply cross validation correctly.
7. Build complete Scikit Learn pipelines that prevent data leakage.

Syllabus

Unit 8.1 The supervised learning framework. Inputs, targets and hypotheses; empirical risk minimisation; loss functions for regression and classification; the generalisation gap; the no free lunch theorem; baselines and why they matter.

Unit 8.2 Linear regression. Ordinary least squares from geometric, optimisation and probabilistic perspectives; assumptions and diagnostics; polynomial features; interpretation of coefficients; gradient descent implementation.

Unit 8.3 Logistic regression and linear classifiers. The sigmoid and softmax functions; cross entropy loss; decision boundaries; multiclass strategies; probability calibration; the perceptron as a historical precursor to neural networks.

Unit 8.4 Generalisation and regularisation. Bias and variance decomposition; learning curves and validation curves; ridge, lasso and elastic net; regularisation as a prior; early stopping; the double descent phenomenon as a modern perspective.

Unit 8.5 Instance based and probabilistic classifiers. k nearest neighbours and the curse of dimensionality; distance metrics and feature scaling; naive Bayes for text classification; generative versus discriminative models.

Unit 8.6 Trees and kernel methods. Decision trees, impurity measures and pruning; support vector machines, margins and hinge loss; the kernel trick with polynomial and radial basis function kernels.

Unit 8.7 Validation and pipelines. Hold out, k fold, stratified and grouped cross validation; time aware splits; nested cross validation for hyperparameter selection; Scikit Learn pipelines and column transformers; common leakage patterns and how to prevent them.

Practical Laboratories

- **Lab 8A: Algorithms from scratch.** Implement linear regression, logistic regression and k nearest neighbours in NumPy with a Scikit Learn compatible interface.
- **Lab 8B: Regularisation paths.** Plot coefficient paths for ridge and lasso on a high dimensional dataset and interpret feature selection.
- **Lab 8C: Leakage detective.** Find and fix five deliberately planted leakage bugs in a modelling notebook.
- **Lab 8D: Credit risk classifier.** Build a full pipeline for a credit default dataset, comparing at least four algorithms with cross validation.

Assessment

Component	Weight	Description
Laboratory portfolio	30%	Labs 8A to 8D.
Level I integrative project	50%	An end to end supervised learning project from raw data to evaluated model, delivered as a tested repository with a technical report.
Viva voce	20%	A twenty minute oral defence of the project covering mathematical and engineering decisions.

Recommended Resources

- Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, An Introduction to Statistical Learning with Applications in Python, Springer.
- Aurelien Geron, Hands On Machine Learning with Scikit Learn, Keras and TensorFlow, O'Reilly.
- The Scikit Learn user guide.

Competency Checkpoint

The learner has completed Level I when they can independently deliver a reproducible supervised learning project that outperforms a sensible baseline, uses leak free validation, reports uncertainty in its metrics, and is defended convincingly in a viva.

Level II: Intermediate Practice

Level II transforms the learner from someone who understands the foundations into a practitioner who can deliver real value. The level broadens classical machine learning into unsupervised methods and powerful ensembles, deepens evaluation and feature engineering, and then introduces deep learning across its three great application domains of vision, language and sequences. It concludes with the first module on MLOps, in which learners take a trained model out of the notebook and turn it into a reliable, containerised service.

By the end of Level II a learner should be employable as a junior machine learning engineer or data scientist. They should be able to choose sensibly between gradient boosted trees and neural networks for a given problem, build and fine tune convolutional and transformer models, and deploy a model behind an application programming interface with tests, logging and monitoring hooks.

> *Level II exit standard: the learner can train a deep learning model on a nontrivial dataset, track experiments systematically, compare it fairly against a strong classical baseline, and deploy the better model as a containerised web service with automated tests.*

Module 9: Unsupervised Learning and Ensembles

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 17 to 18	Level I complete	30 guided and 15 independent

Module Overview

This module covers two families of methods that dominate practical machine learning outside of deep learning. Unsupervised learning finds structure in unlabelled data through clustering, density estimation, dimensionality reduction and anomaly detection. Ensemble methods, particularly random forests and gradient boosted trees, remain the strongest general purpose approach for tabular data and win a large share of applied competitions and production use cases.

Learning Outcomes

By the end of this module the learner will be able to:

1. Implement and apply k means, hierarchical clustering, DBSCAN and Gaussian mixture models.
2. Evaluate clustering results with internal metrics and with domain validation.
3. Apply anomaly detection methods including isolation forests and density based approaches.
4. Explain bagging, boosting and stacking and their effects on bias and variance.

5. Train, tune and interpret random forests and gradient boosted trees using XGBoost, LightGBM and CatBoost.
6. Select between ensembles and other model families on the basis of data characteristics and constraints.

Syllabus

Unit 9.1 Clustering. The k means algorithm and its objective; initialisation with k means plus plus; choosing k with the elbow method, silhouette scores and gap statistics; hierarchical agglomerative clustering and dendrograms; density based clustering with DBSCAN and HDBSCAN.

Unit 9.2 Probabilistic latent variable models. Gaussian mixture models; the expectation maximisation algorithm derived and implemented; soft versus hard assignments; model selection with information criteria.

Unit 9.3 Dimensionality reduction revisited. Kernel PCA; non negative matrix factorisation for parts based representations; autoencoders as a preview of deep learning; manifold learning methods.

Unit 9.4 Anomaly detection. Statistical approaches; isolation forests; local outlier factor; one class support vector machines; evaluation challenges when labels are scarce; applications in fraud, cybersecurity and industrial monitoring.

Unit 9.5 Bagging and random forests. Bootstrap aggregation; feature subsampling; out of bag error; feature importance and its biases; extremely randomised trees.

Unit 9.6 Boosting. AdaBoost; gradient boosting as functional gradient descent; regularisation through shrinkage, subsampling and tree constraints; the engineering innovations in XGBoost, LightGBM and CatBoost including histogram binning, leaf wise growth and ordered target statistics.

Unit 9.7 Stacking and model selection. Stacked generalisation and blending; avoiding leakage in stacking; when ensembles are worth their complexity; inference cost considerations.

Practical Laboratories

- **Lab 9A: Customer segmentation.** Segment retail customers using recency, frequency and monetary features with several clustering algorithms, and profile each segment.
- **Lab 9B: EM from scratch.** Implement expectation maximisation for a Gaussian mixture model and visualise convergence.
- **Lab 9C: Fraud detection.** Detect anomalous card transactions on a highly imbalanced dataset using isolation forests and boosted trees.
- **Lab 9D: Boosting benchmark.** Compare XGBoost, LightGBM and CatBoost on three tabular datasets under equal tuning budgets.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 9A to 9D.
Applied project	45%	An unsupervised or ensemble driven solution to a business problem, with segment or model interpretation.
Written examination	20%	Conceptual questions on clustering, EM and boosting.

Recommended Resources

- Trevor Hastie, Robert Tibshirani and Jerome Friedman, The Elements of Statistical Learning, Springer.
- Tianqi Chen and Carlos Guestrin, XGBoost: A Scalable Tree Boosting System, original research paper.
- The LightGBM and CatBoost documentation.

Competency Checkpoint

The learner is ready to progress when they can tune a gradient boosted model to a strong level on a new tabular dataset, explain the effect of its principal hyperparameters, and defend a segmentation solution to a business audience in terms they understand.

Module 10: Feature Engineering and Model Evaluation

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 19 to 20	Module 9	30 guided and 15 independent

Module Overview

Strong models are built on strong features and judged by appropriate metrics. This module develops systematic feature engineering for numerical, categorical, textual and temporal data, along with a sophisticated understanding of evaluation: choosing metrics that reflect real costs, handling class imbalance, calibrating probabilities, tuning hyperparameters efficiently and explaining model behaviour to stakeholders and regulators.

Learning Outcomes

By the end of this module the learner will be able to:

1. Engineer and select features for diverse data types using sound, leak free techniques.
2. Choose evaluation metrics that align with business costs and justify the choice.

3. Handle class imbalance through resampling, weighting and threshold optimisation.
4. Calibrate predicted probabilities and evaluate calibration quality.
5. Conduct efficient hyperparameter optimisation using random search and Bayesian methods.
6. Explain individual predictions and global behaviour using model agnostic interpretability methods.
7. Perform rigorous error analysis and slice based evaluation.

Syllabus

Unit 10.1 Numerical and categorical features. Scaling and normalisation; power and quantile transforms; binning; interaction features; one hot, ordinal, frequency and target encoding with proper cross fitting; handling high cardinality; hashing tricks.

Unit 10.2 Text, date and domain features. Bag of words and TF IDF; date and time decomposition; cyclical encodings; lag and aggregation features; domain driven feature design through stakeholder interviews.

Unit 10.3 Feature selection. Filter methods using correlation and mutual information; wrapper methods including recursive feature elimination; embedded methods using regularisation and tree importance; permutation importance; stability of selection.

Unit 10.4 Classification metrics in depth. The confusion matrix; precision, recall, specificity and F scores; receiver operating characteristic and precision recall curves; threshold selection based on cost matrices; metrics for multiclass and multilabel problems; ranking metrics.

Unit 10.5 Imbalance and calibration. Random oversampling and undersampling; synthetic minority oversampling; class weights; focal loss; reliability diagrams; Brier score; Platt scaling and isotonic regression.

Unit 10.6 Hyperparameter optimisation. Grid and random search; Bayesian optimisation with Gaussian processes and tree structured Parzen estimators; early stopping strategies such as successive halving and Hyperband; using Optuna; managing the risk of overfitting the validation set.

Unit 10.7 Interpretability and error analysis. Partial dependence and individual conditional expectation plots; Shapley additive explanations; local interpretable model agnostic explanations; counterfactual explanations; slice based evaluation; documenting limitations.

Practical Laboratories

- **Lab 10A: Encoding showdown.** Compare encoding strategies for a high cardinality categorical variable and quantify leakage from naive target encoding.
- **Lab 10B: Cost sensitive thresholds.** Optimise a decision threshold for a hospital readmission model under an explicit cost matrix.
- **Lab 10C: Optuna study.** Tune a LightGBM model with Bayesian optimisation and pruning, and visualise the search.

- **Lab 10D: Explainability report.** Produce global and local explanations for a loan approval model and write a summary for a compliance officer.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 10A to 10D.
Evaluation case study	40%	A full evaluation and interpretation report for a provided model, including error analysis and recommendations.
Written examination	25%	Metric selection, calibration and interpretability reasoning.

Recommended Resources

- Alice Zheng and Amanda Casari, Feature Engineering for Machine Learning, O'Reilly.
- Christoph Molnar, Interpretable Machine Learning, freely available online.
- Max Kuhn and Kjell Johnson, Feature Engineering and Selection, CRC Press.

Competency Checkpoint

The learner is ready to progress when they can take a trained model, choose and justify the right metric for its business context, set an optimal threshold, demonstrate calibration and produce explanations suitable for a regulator.

Module 11: Neural Network Foundations with PyTorch

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 21 to 22	Modules 3, 8 and 10	30 guided and 15 independent

Module Overview

This module marks the transition to deep learning. Learners first build a multilayer perceptron entirely from scratch in NumPy, including manual backpropagation, so that every component is understood. They then learn PyTorch in depth, mastering tensors, automatic differentiation, modules, data loaders and training loops. The module concludes with the practical craft of training deep networks: initialisation, normalisation, regularisation, learning rate schedules and systematic debugging.

Learning Outcomes

By the end of this module the learner will be able to:

1. Explain the universal approximation property and the representational benefits of depth.

2. Implement forward and backward passes of a multilayer perceptron from scratch.
3. Build, train and evaluate neural networks in PyTorch using idiomatic patterns.
4. Select appropriate activation functions, initialisation schemes and normalisation layers.
5. Apply dropout, weight decay, data augmentation and early stopping to control overfitting.
6. Diagnose and fix common training failures such as vanishing gradients, divergence and silent bugs.
7. Use graphics processor acceleration and mixed precision training effectively.

Syllabus

Unit 11.1 From perceptrons to deep networks. The artificial neuron; multilayer perceptrons; activation functions including sigmoid, tanh, ReLU, GELU and SiLU; universal approximation; why depth helps; the history of deep learning and its recent acceleration.

Unit 11.2 Backpropagation in detail. Deriving gradients for dense layers, activations and softmax cross entropy; vectorised backpropagation over mini batches; implementing a complete network in NumPy and verifying gradients numerically.

Unit 11.3 PyTorch fundamentals. Tensors and devices; autograd and the dynamic computational graph; nn Module and parameter registration; optimisers; datasets and data loaders; writing clean training and evaluation loops; saving and loading checkpoints.

Unit 11.4 Initialisation and normalisation. Xavier and He initialisation; batch normalisation, layer normalisation and group normalisation; residual connections and their role in enabling very deep networks.

Unit 11.5 Regularisation and generalisation. Weight decay and its interaction with adaptive optimisers; dropout; data augmentation; label smoothing; early stopping; the surprising generalisation of overparameterised networks.

Unit 11.6 Training dynamics. Learning rate finders; warmup and cosine decay schedules; one cycle policy; batch size effects; gradient clipping; mixed precision with automatic loss scaling; reproducibility and determinism.

Unit 11.7 Debugging neural networks. A systematic recipe: overfit a single batch, verify loss at initialisation, visualise activations and gradients, check data pipelines; using TensorBoard; PyTorch Lightning for structured training code.

Practical Laboratories

- **Lab 11A: NumPy network.** Build and train a two hidden layer network on handwritten digits without any deep learning framework.
- **Lab 11B: PyTorch port.** Reimplement the same network idiomatically in PyTorch and compare speed on processor and graphics processor.

- **Lab 11C: Ablation study.** Measure the effect of initialisation, normalisation, dropout and learning rate schedules on a tabular classification task.
- **Lab 11D: Bug hunt.** Find and fix ten planted bugs in a broken PyTorch training script.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 11A to 11D.
Deep learning assignment	45%	Train a neural network on a tabular or signal dataset and compare it fairly against a gradient boosted baseline.
Viva voce	20%	Explanation of backpropagation and the learner's training decisions.

Recommended Resources

- Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press.
- Simon Prince, Understanding Deep Learning, MIT Press.
- Aston Zhang and colleagues, Dive into Deep Learning, freely available online.
- Andrej Karpathy, A Recipe for Training Neural Networks, essay.

Competency Checkpoint

The learner is ready to progress when they can write a complete PyTorch training pipeline from a blank file, explain every line, and systematically debug a network that refuses to learn.

Module 12: Computer Vision with Deep Learning

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 23 to 24	Module 11	30 guided and 15 independent

Module Overview

Computer vision was the field in which deep learning first demonstrated overwhelming superiority, and it remains one of the most commercially important application areas. This module covers convolutional neural networks from first principles, landmark architectures, transfer learning, object detection, segmentation and vision transformers. Practical work emphasises data pipelines, augmentation and deployment considerations such as model size and latency on edge devices.

Learning Outcomes

By the end of this module the learner will be able to:

1. Explain convolution, pooling, receptive fields and parameter sharing.
2. Describe the design principles behind landmark architectures from LeNet to ConvNeXt.
3. Apply transfer learning and fine tuning to small domain specific image datasets.
4. Train and evaluate object detection and semantic segmentation models.
5. Explain how vision transformers and self supervised methods learn visual representations.
6. Interpret vision models with saliency and class activation mapping techniques.
7. Optimise vision models for deployment under latency and memory constraints.

Syllabus

Unit 12.1 Image fundamentals. Digital images as tensors; colour spaces; classical image processing including filtering and edge detection; why fully connected networks fail on images.

Unit 12.2 Convolutional neural networks. The convolution operation; padding, stride and dilation; pooling; receptive fields; computing output shapes and parameter counts; implementing a convolution from scratch.

Unit 12.3 Landmark architectures. LeNet, AlexNet, VGG, Inception, ResNet, DenseNet, MobileNet, EfficientNet and ConvNeXt; design patterns including bottlenecks, depthwise separable convolutions and compound scaling.

Unit 12.4 Transfer learning and augmentation. Pretrained backbones; feature extraction versus fine tuning; discriminative learning rates; augmentation strategies including random cropping, colour jitter, Mixup, CutMix and RandAugment; handling small and imbalanced image datasets.

Unit 12.5 Detection and segmentation. Bounding boxes, intersection over union and mean average precision; two stage detectors such as Faster R CNN; single stage detectors such as YOLO and RetinaNet; semantic segmentation with U Net and DeepLab; instance segmentation with Mask R CNN; foundation segmentation models.

Unit 12.6 Vision transformers and self supervision. Patch embeddings and the vision transformer; contrastive learning with SimCLR; masked autoencoders; vision language models such as CLIP and zero shot classification.

Unit 12.7 Interpretation and deployment. Saliency maps, Grad CAM and occlusion analysis; adversarial examples; model compression; exporting to ONNX; inference on edge devices.

Practical Laboratories

- **Lab 12A: CNN from scratch.** Train a small convolutional network on a standard benchmark and visualise learned filters.

- **Lab 12B: Transfer learning.** Fine tune a pretrained ResNet on a medical or agricultural image dataset with fewer than two thousand images.
- **Lab 12C: Detection pipeline.** Train an object detector for a safety equipment detection task and report mean average precision.
- **Lab 12D: Zero shot with CLIP.** Compare zero shot CLIP classification against a fine tuned model and analyse failure cases.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 12A to 12D.
Vision project	50%	An applied vision system addressing a real problem, including data curation, training, evaluation and an interpretability analysis.
Presentation	15%	Demonstration of the system with discussion of limitations and risks.

Recommended Resources

- Stanford University, CS231n Convolutional Neural Networks for Visual Recognition, course notes.
- Kaiming He and colleagues, Deep Residual Learning for Image Recognition, research paper.
- Alexey Dosovitskiy and colleagues, An Image is Worth 16x16 Words, research paper.

Competency Checkpoint

The learner is ready to progress when they can take a new image classification or detection problem with limited data and deliver a well validated, interpretable model using transfer learning, with a clear account of its failure modes.

Module 13: Natural Language Processing and Sequence Models

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 25 to 26	Module 11	30 guided and 15 independent

Module Overview

Language is the richest and most ambiguous form of data that machine learning engineers encounter. This module traces the evolution of natural language processing from rule based and statistical methods through word embeddings and recurrent neural networks to the sequence to sequence models that set the

stage for transformers. Understanding this progression gives learners the conceptual depth needed to work effectively with modern language models and to recognise when simpler methods are sufficient.

Learning Outcomes

By the end of this module the learner will be able to:

1. Preprocess and tokenise text using classical and subword tokenisation methods.
2. Build strong classical baselines for text classification using TF IDF and linear models.
3. Explain and train word embeddings and use them for semantic similarity.
4. Implement recurrent neural networks, LSTMs and GRUs and explain their gradient dynamics.
5. Build sequence to sequence models with attention for translation or summarisation.
6. Evaluate language tasks using appropriate automatic and human evaluation methods.

Syllabus

Unit 13.1 Text processing fundamentals. Normalisation, tokenisation, stemming and lemmatisation; regular expressions; part of speech tagging and named entity recognition with spaCy; the challenges of multilingual and informal text.

Unit 13.2 Statistical language modelling. N gram language models; smoothing; perplexity; bag of words and TF IDF representations; naive Bayes and logistic regression for sentiment and topic classification; topic modelling with latent Dirichlet allocation.

Unit 13.3 Word embeddings. The distributional hypothesis; word2vec skip gram and continuous bag of words; negative sampling; GloVe and fastText; analogies and their limitations; bias in embeddings.

Unit 13.4 Recurrent neural networks. Sequence modelling with recurrence; backpropagation through time; vanishing and exploding gradients; long short term memory and gated recurrent units; bidirectional and stacked recurrent networks; character level language models.

Unit 13.5 Sequence to sequence and attention. Encoder decoder architectures; teacher forcing; beam search decoding; Bahdanau and Luong attention; the information bottleneck that attention resolves.

Unit 13.6 Subword tokenisation. Byte pair encoding, WordPiece and unigram language model tokenisers; SentencePiece; vocabulary size trade offs; tokenisation artefacts and their downstream consequences.

Unit 13.7 Evaluation of language systems. Accuracy and F1 for classification; BLEU, ROUGE, METEOR and BERTScore for generation; their correlation with human judgement; designing human evaluation studies.

Practical Laboratories

- **Lab 13A: Classical baseline.** Build a TF IDF and logistic regression classifier for customer complaint routing that later deep models must beat.

- **Lab 13B: Embeddings explorer.** Train word2vec on a domain corpus and probe it for analogies and biases.
- **Lab 13C: Character LSTM.** Train a character level LSTM to generate text in the style of a public domain author.
- **Lab 13D: Neural translation.** Build a sequence to sequence model with attention for a small translation task and visualise attention weights.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 13A to 13D.
NLP assignment	45%	A text classification or extraction system comparing classical, embedding based and recurrent approaches.
Written examination	20%	Questions on embeddings, recurrence, attention and evaluation.

Recommended Resources

- Daniel Jurafsky and James H. Martin, Speech and Language Processing, draft third edition, freely available online.
- Stanford University, CS224n Natural Language Processing with Deep Learning, course materials.
- Yoav Goldberg, Neural Network Methods for Natural Language Processing, Morgan and Claypool.

Competency Checkpoint

The learner is ready to progress when they can build a strong classical text baseline in under an hour, explain precisely why recurrent networks struggle with long range dependencies, and describe how attention addresses that weakness.

Module 14: Attention and the Transformer Architecture

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 27 to 28	Modules 11 and 13	30 guided and 15 independent

Module Overview

The transformer is the most consequential neural architecture of the past decade. It underpins large language models, modern vision systems, speech recognition, protein structure prediction and much more. This module dissects the transformer completely. Learners implement scaled dot product attention,

multi head attention, positional encodings and full encoder and decoder blocks from scratch, then train a small generative language model. They then learn to use pretrained transformers through the Hugging Face ecosystem for classification, extraction and generation.

Learning Outcomes

By the end of this module the learner will be able to:

1. Derive and implement scaled dot product attention and multi head attention.
2. Explain the role of positional information, residual streams, layer normalisation and feedforward blocks.
3. Distinguish encoder only, decoder only and encoder decoder transformer families and their use cases.
4. Implement and train a small decoder only transformer for language modelling.
5. Fine tune pretrained transformers for classification, token classification and question answering.
6. Analyse the computational complexity of attention and describe efficient attention variants.

Syllabus

Unit 14.1 Self attention. Queries, keys and values; scaled dot product attention derived; masking for causal modelling and padding; multi head attention as parallel subspace attention; implementing attention in fewer than thirty lines of PyTorch.

Unit 14.2 The transformer block. Residual connections and the residual stream view; pre normalisation versus post normalisation; position wise feedforward networks; activation choices such as GELU and SwiGLU; parameter counting.

Unit 14.3 Positional information. Sinusoidal encodings; learned positional embeddings; relative position methods; rotary position embeddings; length generalisation and context extension.

Unit 14.4 Architectural families. Encoder only models such as BERT and RoBERTa and their masked language modelling objective; decoder only models such as the GPT family and their causal objective; encoder decoder models such as T5 and BART; choosing a family for a task.

Unit 14.5 Building a small language model. Tokenisation, dataset preparation and batching; training a decoder only transformer on a modest corpus; sampling strategies including greedy, temperature, top k and nucleus sampling; evaluating with perplexity.

Unit 14.6 The Hugging Face ecosystem. Transformers, Tokenizers, Datasets, Evaluate and Accelerate libraries; the Trainer interface; model cards; fine tuning for sequence classification, named entity recognition and extractive question answering.

Unit 14.7 Efficiency and interpretability. Quadratic complexity of attention; sparse, linear and windowed attention; FlashAttention and memory efficient kernels; key value caching; mechanistic interpretability concepts such as attention heads, circuits and induction heads.

Practical Laboratories

- **Lab 14A: Attention from scratch.** Implement and unit test scaled dot product and multi head attention against reference outputs.
- **Lab 14B: Mini GPT.** Train a small decoder only transformer on a public domain text corpus and experiment with sampling strategies.
- **Lab 14C: BERT fine tuning.** Fine tune a pretrained encoder for clinical or legal text classification and compare with the Module 13 baselines.
- **Lab 14D: Attention visualisation.** Visualise and interpret attention patterns in a pretrained model.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 14A to 14D.
Transformer implementation	40%	A documented from scratch transformer implementation with training results and an ablation of at least two design choices.
Viva voce	25%	An oral examination on attention, architecture choices and complexity.

Recommended Resources

- Ashish Vaswani and colleagues, Attention Is All You Need, research paper.
- Lewis Tunstall, Leandro von Werra and Thomas Wolf, Natural Language Processing with Transformers, O'Reilly.
- Jay Alammar, The Illustrated Transformer, online article.
- Andrej Karpathy, the nanoGPT repository and associated lecture.

Competency Checkpoint

The learner is ready to progress when they can write a working transformer block from memory, explain every tensor shape through the forward pass, and fine tune a pretrained model to beat a classical baseline on a domain task.

Module 15: Time Series Forecasting and Recommender Systems

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 29 to 30	Modules 9, 10 and 11	30 guided and 15 independent

Module Overview

Two of the most commercially valuable applications of machine learning are forecasting and recommendation. Forecasting drives inventory, staffing, energy and financial planning, while recommender systems drive engagement and revenue on almost every digital platform. Both domains have distinctive data structures, evaluation challenges and modelling approaches that are poorly served by generic machine learning training. This module addresses that gap.

Learning Outcomes

By the end of this module the learner will be able to:

1. Decompose and analyse time series for trend, seasonality, stationarity and autocorrelation.
2. Build statistical forecasting models including exponential smoothing and ARIMA.
3. Apply machine learning and deep learning approaches to forecasting with correct temporal validation.
4. Produce and evaluate probabilistic forecasts and prediction intervals.
5. Build collaborative filtering, content based and hybrid recommender systems.
6. Evaluate recommenders with offline ranking metrics and explain the role of online experimentation.

Syllabus

Unit 15.1 Time series foundations. Components of a time series; stationarity and differencing; autocorrelation and partial autocorrelation functions; seasonal decomposition; handling missing timestamps and irregular sampling.

Unit 15.2 Statistical forecasting. Naive and seasonal naive benchmarks; exponential smoothing and the ETS framework; ARIMA and seasonal ARIMA; exogenous regressors; hierarchical forecasting and reconciliation.

Unit 15.3 Machine learning for forecasting. Framing forecasting as supervised learning; lag, rolling window and calendar features; global models across many series using gradient boosting; rolling origin evaluation and backtesting; avoiding lookahead leakage.

Unit 15.4 Deep and probabilistic forecasting. Recurrent and convolutional forecasters; DeepAR; N BEATS; temporal fusion transformers; foundation models for time series; quantile loss; conformal prediction intervals; scoring rules such as the continuous ranked probability score.

Unit 15.5 Recommender foundations. Explicit and implicit feedback; the user item matrix; popularity and content based approaches; neighbourhood based collaborative filtering; the cold start problem.

Unit 15.6 Matrix factorisation and neural recommenders. Matrix factorisation with alternating least squares and stochastic gradient descent; implicit feedback models; two tower retrieval architectures; sequential recommenders; candidate generation and ranking stages in industrial systems.

Unit 15.7 Evaluating recommenders. Precision and recall at k; mean average precision; normalised discounted cumulative gain; coverage, diversity, novelty and serendipity; offline and online evaluation gaps; feedback loops and popularity bias.

Practical Laboratories

- **Lab 15A: Retail demand forecasting.** Forecast daily sales for hundreds of products comparing ETS, ARIMA and a global LightGBM model with backtesting.
- **Lab 15B: Probabilistic forecasts.** Produce quantile forecasts and conformal intervals for electricity demand and assess coverage.
- **Lab 15C: Movie recommender.** Build and compare collaborative filtering and matrix factorisation recommenders on a public ratings dataset.
- **Lab 15D: Two tower retrieval.** Implement a two tower model for implicit feedback and evaluate with ranking metrics.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 15A to 15D.
Applied project	50%	Either a forecasting or a recommendation system for a realistic business scenario, with rigorous evaluation.
Written examination	15%	Questions on temporal validation, forecasting models and ranking metrics.

Recommended Resources

- Rob J. Hyndman and George Athanasopoulos, Forecasting: Principles and Practice, freely available online.
- Charu Aggarwal, Recommender Systems: The Textbook, Springer.
- Kim Falk, Practical Recommender Systems, Manning.

Competency Checkpoint

The learner is ready to progress when they can design a backtesting scheme that a sceptical finance director would trust, and explain why offline ranking metrics alone are insufficient to launch a recommender.

Module 16: MLOps I: From Notebook to Service

Level	Schedule	Prerequisites	Study Hours
Level II Intermediate	Weeks 31 to 32	Modules 7, 10 and 11	30 guided and 15 independent

Module Overview

A model that cannot be reproduced, deployed and monitored delivers no value. This module introduces machine learning operations, the discipline of reliably delivering machine learning into production. Learners track experiments, version data and models, package models as services behind application programming interfaces, containerise them with Docker, and automate testing and deployment. The module concludes the intermediate level with a fully deployed, containerised model service.

Learning Outcomes

By the end of this module the learner will be able to:

1. Explain the machine learning lifecycle and the sources of technical debt specific to machine learning systems.
2. Track experiments, parameters, metrics and artefacts using MLflow or Weights and Biases.
3. Version datasets and models and register models with lifecycle stages.
4. Serve models through a typed, validated and documented REST interface using FastAPI.
5. Containerise applications with Docker following security and size best practices.
6. Build continuous integration and delivery pipelines for machine learning applications.
7. Instrument services with structured logging, metrics and health checks.

Syllabus

Unit 16.1 The machine learning lifecycle. Problem framing, data, modelling, deployment and monitoring as a continuous loop; hidden technical debt in machine learning systems; MLOps maturity levels; roles in machine learning teams.

Unit 16.2 Experiment tracking and reproducibility. Logging parameters, metrics and artefacts; comparing runs; seeding and determinism; environment capture; configuration management with Hydra.

Unit 16.3 Data and model versioning. Versioning data with DVC; model registries; model cards and metadata; promoting models through staging and production; rollback strategies.

Unit 16.4 Model serving. Batch, online and streaming inference patterns; building prediction services with FastAPI; request and response validation with Pydantic; serialisation formats including pickle, joblib, ONNX and TorchScript, with their security implications; latency budgets.

Unit 16.5 Containerisation. Docker images, layers and caching; writing efficient Dockerfiles; multi stage builds; running as a non root user; image scanning; Docker Compose for local multi service development.

Unit 16.6 Continuous integration and delivery for ML. Testing models, data and code; smoke tests and contract tests; automated retraining triggers; deployment strategies including blue green, canary and shadow deployment.

Unit 16.7 Observability foundations. Structured logging; application metrics with Prometheus; dashboards with Grafana; health and readiness endpoints; logging predictions for later monitoring and retraining.

Practical Laboratories

- **Lab 16A: Tracked experiments.** Retrofit experiment tracking and a model registry to a previous module project.
- **Lab 16B: Prediction API.** Serve the registered model through FastAPI with validation, documentation and tests.
- **Lab 16C: Containerise and compose.** Containerise the service and run it with a monitoring stack using Docker Compose.
- **Lab 16D: Delivery pipeline.** Build a workflow that tests, builds and publishes the container image on every merge to the main branch.

Assessment

Component	Weight	Description
Laboratory portfolio	30%	Labs 16A to 16D.
Level II integrative project	50%	A deployed, containerised and monitored model service built on a model from Modules 12 to 15.
Design review	20%	A presentation of the system architecture and its operational risks.

Recommended Resources

- Chip Huyen, Designing Machine Learning Systems, O'Reilly.
- D. Sculley and colleagues, Hidden Technical Debt in Machine Learning Systems, research paper.
- Noah Gift and Alfredo Deza, Practical MLOps, O'Reilly.

Competency Checkpoint

The learner has completed Level II when another engineer can clone their repository, run a single command to start the service, retrieve a prediction, and trace that prediction back to the exact data, code and parameters that produced the model.

Level III: Advanced Engineering

Level III develops the capabilities that distinguish senior machine learning engineers, AI engineers and applied research scientists. It addresses the frontier of current practice: large language models and their adaptation, retrieval augmented generation and autonomous agents, deep generative models, reinforcement learning, distributed training and efficient inference, production systems at scale, responsible AI and governance, and the art of designing complete machine learning systems under real constraints.

The level assumes complete command of Levels I and II. The pace is faster, the reading is drawn increasingly from primary research literature, and laboratories are more open ended. Learners are expected to make and justify design decisions with less scaffolding, to read and reproduce results from research papers, and to consider cost, safety and governance alongside accuracy in every piece of work.

> *Level III exit standard: the learner can design, build, evaluate, deploy and govern a production grade AI system involving foundation models or advanced learning methods, defend every design decision before a panel of senior engineers, and communicate its risks honestly to non technical stakeholders.*

Module 17: Large Language Models: Pretraining and Adaptation

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 33 to 34	Level II complete	30 guided and 15 independent

Module Overview

Large language models have redefined what is possible with software. This module explains how they are built and how they are adapted. Learners study the full pipeline from data curation and pretraining through supervised instruction tuning and preference alignment, then gain hands on experience with parameter efficient fine tuning, quantisation, prompting strategies and rigorous evaluation. The module balances conceptual depth with the practical judgement required to decide when to prompt, when to retrieve and when to fine tune.

Learning Outcomes

By the end of this module the learner will be able to:

1. Describe the end to end lifecycle of a large language model from data curation to alignment.
2. Explain scaling laws and their implications for model size, data volume and compute budgets.
3. Apply prompt engineering techniques systematically and measure their effect.

4. Fine tune open weight models using full fine tuning and parameter efficient methods such as LoRA and QLoRA.
5. Explain instruction tuning and preference optimisation methods including RLHF and DPO.
6. Quantise models and evaluate the trade offs between quality, memory and latency.
7. Design robust evaluation suites for generative language systems, including model based evaluation.

Syllabus

Unit 17.1 Pretraining. Data sources, deduplication, quality filtering and contamination; tokeniser training; the causal language modelling objective at scale; training stability techniques; mixture of experts architectures; the economics of pretraining compute.

Unit 17.2 Scaling laws and emergent behaviour. Empirical scaling laws for loss as a function of parameters, data and compute; compute optimal training; the debate over emergent abilities; implications for capability forecasting.

Unit 17.3 Prompting and in context learning. Zero shot and few shot prompting; instruction design; chain of thought and other reasoning elicitation methods; structured outputs and function calling; prompt robustness and sensitivity testing; systematic prompt versioning and evaluation.

Unit 17.4 Supervised fine tuning. Instruction datasets and their construction; chat templates; full fine tuning versus parameter efficient approaches; low rank adaptation explained mathematically; QLoRA with four bit quantised base models; adapters and prefix tuning; catastrophic forgetting.

Unit 17.5 Alignment and preference optimisation. Reward modelling; reinforcement learning from human feedback with proximal policy optimisation; direct preference optimisation and related methods; constitutional and AI feedback approaches; reasoning models trained with reinforcement learning on verifiable rewards.

Unit 17.6 Quantisation and compression. Floating point formats; post training quantisation; GPTQ, AWQ and related methods; distillation of large models into small ones; pruning; measuring quality degradation.

Unit 17.7 Evaluating language models. Benchmark suites and their limitations; contamination; task specific evaluation sets; model graded evaluation and its biases; human preference evaluation; hallucination measurement; building an evaluation harness that runs in continuous integration.

Practical Laboratories

- **Lab 17A: Prompt engineering study.** Design, version and evaluate prompts for a structured information extraction task with a held out test set.
- **Lab 17B: LoRA fine tuning.** Fine tune a small open weight model with QLoRA for a domain specific instruction following task.

- **Lab 17C: Preference tuning.** Apply direct preference optimisation using a small preference dataset and measure behavioural change.
- **Lab 17D: Quantisation trade offs.** Quantise a model at several precisions and chart quality, memory and throughput.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 17A to 17D.
Adaptation project	45%	Adapt an open weight model to a specialised domain and demonstrate improvement over prompting alone with a rigorous evaluation harness.
Research paper review	20%	A critical review and partial reproduction of a recent paper on language model adaptation.

Recommended Resources

- Sebastian Raschka, Build a Large Language Model from Scratch, Manning.
- Chip Huyen, AI Engineering, O'Reilly.
- Jared Kaplan and colleagues, Scaling Laws for Neural Language Models, and Jordan Hoffmann and colleagues, Training Compute Optimal Large Language Models, research papers.
- Edward Hu and colleagues, LoRA: Low Rank Adaptation of Large Language Models, research paper.

Competency Checkpoint

The learner is ready to progress when, faced with a new business use case, they can justify whether prompting, retrieval or fine tuning is appropriate, implement the chosen approach, and prove its value with a trustworthy evaluation.

Module 18: Retrieval Augmented Generation and AI Agents

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 35 to 36	Module 17	30 guided and 15 independent

Module Overview

Most enterprise applications of language models depend on connecting them to private knowledge and to external tools. Retrieval augmented generation grounds model outputs in authoritative documents, reducing hallucination and enabling citation. Agents extend models with the ability to plan, call tools and

act. This module develops production grade competence in both, with strong emphasis on retrieval quality, evaluation, latency, cost, security and the failure modes that cause real deployments to fail.

Learning Outcomes

By the end of this module the learner will be able to:

1. Design and implement end to end retrieval augmented generation pipelines.
2. Select and tune chunking strategies, embedding models, vector indexes and rerankers.
3. Implement hybrid retrieval combining lexical and dense search.
4. Evaluate retrieval and generation quality separately and jointly using appropriate metrics.
5. Build tool using agents with planning, memory and error recovery.
6. Implement guardrails against prompt injection, data leakage and unsafe actions.
7. Optimise retrieval augmented and agentic systems for latency and cost.

Syllabus

Unit 18.1 Embeddings and vector search. Sentence embedding models and how they are trained; similarity measures; approximate nearest neighbour algorithms including HNSW and inverted file indexes; product quantisation; vector databases and their operational trade offs.

Unit 18.2 Ingestion and chunking. Document parsing for PDF, HTML and office formats; table and image handling; fixed, recursive, semantic and structure aware chunking; metadata enrichment; incremental indexing and freshness.

Unit 18.3 Advanced retrieval. BM25 and hybrid search; reciprocal rank fusion; cross encoder reranking; query rewriting and expansion; hypothetical document embeddings; multi hop retrieval; knowledge graphs and graph based retrieval.

Unit 18.4 Generation and grounding. Context construction and ordering; long context models versus retrieval; citation and attribution; handling conflicting sources; refusing when evidence is insufficient.

Unit 18.5 Evaluation of RAG systems. Retrieval metrics such as recall at k and mean reciprocal rank; generation metrics such as faithfulness, answer relevance and context precision; constructing golden question sets; model graded evaluation frameworks; regression testing.

Unit 18.6 Agents and tool use. Function calling; the reason and act pattern; planning and task decomposition; memory architectures; the Model Context Protocol and tool interoperability; multi agent orchestration; when agents are and are not appropriate.

Unit 18.7 Security, safety and operations. Direct and indirect prompt injection; data exfiltration risks; least privilege tool design; human in the loop approval for consequential actions; tracing and observability for agent workflows; caching, routing and cost control.

Practical Laboratories

- **Lab 18A: Baseline RAG.** Build a question answering system over a corpus of policy documents with citations.
- **Lab 18B: Retrieval optimisation.** Improve retrieval with hybrid search, reranking and chunking experiments, measuring each change.
- **Lab 18C: Evaluation harness.** Construct a golden dataset and automated evaluation pipeline for the RAG system.
- **Lab 18D: Tool using agent.** Build an agent that answers analytical questions by querying a SQL database and a document store, with guardrails and tracing.

Assessment

Component	Weight	Description
Laboratory portfolio	30%	Labs 18A to 18D.
RAG or agent product	50%	A deployed knowledge assistant or agent for a realistic organisation, with evaluation results, security analysis and cost model.
Red team exercise	20%	An adversarial assessment of a peer's system, with a written report of findings and mitigations.

Recommended Resources

- Patrick Lewis and colleagues, Retrieval Augmented Generation for Knowledge Intensive NLP Tasks, research paper.
- Shunyu Yao and colleagues, ReAct: Synergizing Reasoning and Acting in Language Models, research paper.
- The OWASP Top 10 for Large Language Model Applications.
- Chip Huyen, AI Engineering, chapters on RAG and agents.

Competency Checkpoint

The learner is ready to progress when they can show, with quantitative evidence, that each component of their retrieval augmented system contributes to quality, and can demonstrate resilience against a documented set of prompt injection attacks.

Module 19: Deep Generative Models

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 37 to 38	Modules 4, 11 and 12	30 guided and 15 independent

Module Overview

Generative models learn the distribution of data well enough to produce new samples from it. They power image and audio synthesis, data augmentation, drug discovery, anomaly detection and simulation. This module develops a unified probabilistic understanding of the major families of deep generative models, namely autoregressive models, variational autoencoders, generative adversarial networks, normalising flows and diffusion models, and gives learners practical experience training and evaluating them.

Learning Outcomes

By the end of this module the learner will be able to:

1. Explain the taxonomy of generative models by how they represent and approximate likelihood.
2. Derive the evidence lower bound and implement a variational autoencoder.
3. Explain the adversarial training game, its instabilities and stabilisation techniques.
4. Derive the forward and reverse processes of denoising diffusion models and implement a simple sampler.
5. Apply conditioning and guidance techniques for controllable generation.
6. Evaluate generative models using quantitative metrics and structured human assessment.
7. Analyse ethical risks of generative media including deepfakes and copyright concerns.

Syllabus

Unit 19.1 Foundations of generative modelling. Density estimation; explicit versus implicit models; tractable and approximate likelihoods; the trade offs between sample quality, diversity and speed.

Unit 19.2 Autoregressive models. Factorising joint distributions; PixelCNN and WaveNet; autoregressive transformers for images and audio; discrete tokenisation of continuous signals.

Unit 19.3 Variational autoencoders. Latent variable models; variational inference; the evidence lower bound derived; the reparameterisation trick; posterior collapse; beta VAE and disentanglement; vector quantised VAEs.

Unit 19.4 Generative adversarial networks. The minimax objective; mode collapse and training instability; Wasserstein GANs and gradient penalties; conditional GANs; StyleGAN architecture; image to image translation.

Unit 19.5 Normalising flows. Change of variables; invertible architectures including coupling layers; exact likelihood computation; flow matching as a modern bridge to diffusion.

Unit 19.6 Diffusion models. The forward noising process; the reverse denoising process; the simplified training objective; score based perspectives; samplers and acceleration; classifier free guidance; latent diffusion and text to image systems; diffusion transformers.

Unit 19.7 Evaluation and responsibility. Frechet Inception Distance, precision and recall for generative models, CLIP scores; human evaluation protocols; watermarking and provenance standards; deepfake detection; consent and intellectual property considerations.

Practical Laboratories

- **Lab 19A: VAE latent spaces.** Train a variational autoencoder on images and explore interpolation and latent arithmetic.
- **Lab 19B: Taming a GAN.** Train a DCGAN, diagnose mode collapse and apply stabilisation techniques.
- **Lab 19C: Diffusion from scratch.** Implement a denoising diffusion probabilistic model on a small image dataset.
- **Lab 19D: Controlled generation.** Fine tune a latent diffusion model on a small custom dataset with low rank adaptation and evaluate outputs.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 19A to 19D.
Generative project	45%	A generative modelling project for synthetic data or creative application, with quantitative evaluation and an ethics statement.
Written examination	20%	Derivations of the evidence lower bound and diffusion objectives, and conceptual comparisons.

Recommended Resources

- David Foster, Generative Deep Learning, O'Reilly.
- Jonathan Ho, Ajay Jain and Pieter Abbeel, Denoising Diffusion Probabilistic Models, research paper.
- Diederik Kingma and Max Welling, Auto Encoding Variational Bayes, research paper.
- Simon Prince, Understanding Deep Learning, chapters on generative models.

Competency Checkpoint

The learner is ready to progress when they can explain on a whiteboard how a diffusion model is trained and sampled, relate it to variational methods, and implement a working sampler from a blank file.

Module 20: Reinforcement Learning

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 39 to 40	Modules 3, 4 and 11	30 guided and 15 independent

Module Overview

Reinforcement learning addresses sequential decision making, in which an agent learns by interacting with an environment and receiving rewards. It underpins game playing systems, robotics, resource allocation, recommendation policies and, increasingly, the alignment and reasoning training of language models. This module builds from the mathematical framework of Markov decision processes through tabular methods to deep value based and policy gradient algorithms, with a practical emphasis on implementation, debugging and responsible application.

Learning Outcomes

By the end of this module the learner will be able to:

1. Formulate problems as Markov decision processes with appropriate states, actions and rewards.
2. Derive and apply the Bellman equations and dynamic programming methods.
3. Implement Monte Carlo, temporal difference, SARSA and Q learning algorithms.
4. Implement deep Q networks and explain the stabilising role of replay buffers and target networks.
5. Derive the policy gradient theorem and implement REINFORCE, actor critic and proximal policy optimisation.
6. Explain multi armed bandits and apply them to online experimentation.
7. Identify risks such as reward hacking and design safer reward specifications.

Syllabus

Unit 20.1 Bandits and exploration. The exploration and exploitation dilemma; epsilon greedy, upper confidence bound and Thompson sampling; contextual bandits; applications in recommendation and adaptive experimentation.

Unit 20.2 Markov decision processes. States, actions, transitions and rewards; returns and discounting; policies and value functions; the Bellman expectation and optimality equations; policy iteration and value iteration.

Unit 20.3 Model free prediction and control. Monte Carlo methods; temporal difference learning; SARSA and Q learning; eligibility traces; on policy versus off policy learning.

Unit 20.4 Function approximation and deep Q learning. Linear and neural value function approximation; the deadly triad; deep Q networks; experience replay; target networks; double, dueling and prioritised variants.

Unit 20.5 Policy gradient methods. The policy gradient theorem derived; REINFORCE and variance reduction with baselines; advantage actor critic; trust region methods; proximal policy optimisation in detail; continuous action spaces with soft actor critic.

Unit 20.6 Advanced topics. Model based reinforcement learning and planning; offline reinforcement learning from logged data; imitation learning and inverse reinforcement learning; multi agent settings; reinforcement learning for language model alignment and reasoning.

Unit 20.7 Practice and safety. Environment design with Gymnasium; reward shaping and reward hacking; reproducibility challenges and evaluation across seeds; simulation to reality transfer; safe exploration.

Practical Laboratories

- **Lab 20A: Bandit arena.** Compare exploration strategies on simulated advertising data and plot cumulative regret.
- **Lab 20B: Gridworld planning.** Implement value iteration, policy iteration and Q learning on custom gridworlds.
- **Lab 20C: Deep Q network.** Train a deep Q network on a classic control environment and conduct a stabilisation ablation.
- **Lab 20D: Proximal policy optimisation.** Implement proximal policy optimisation from scratch and benchmark against a reference library across multiple seeds.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 20A to 20D.
RL project	45%	Apply reinforcement learning to an operational problem such as inventory control, energy scheduling or traffic signals, with a comparison against heuristic baselines.
Written examination	20%	Bellman equations, policy gradient derivation and algorithm analysis.

Recommended Resources

- Richard Sutton and Andrew Barto, Reinforcement Learning: An Introduction, MIT Press, freely available online.
- OpenAI, Spinning Up in Deep Reinforcement Learning, online resource.
- John Schulman and colleagues, Proximal Policy Optimization Algorithms, research paper.

Competency Checkpoint

The learner is ready to progress when they can implement a working policy gradient agent, diagnose why it fails to learn on a new environment, and articulate whether reinforcement learning is genuinely the right tool for a proposed business problem.

Module 21: Distributed Training and Efficient Inference

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 41 to 42	Modules 11, 14 and 17	30 guided and 15 independent

Module Overview

Modern models are often too large to train on a single accelerator and too expensive to serve naively. This module teaches the systems knowledge that separates engineers who can work at scale from those who cannot. Learners study accelerator hardware, profiling, parallelism strategies, memory optimisation and high throughput inference serving. The focus is on understanding the bottlenecks of compute, memory and communication, and making principled trade offs among them.

Learning Outcomes

By the end of this module the learner will be able to:

1. Explain the architecture of graphics processors and other accelerators and their performance characteristics.
2. Profile training and inference workloads to identify compute, memory and input bottlenecks.
3. Apply data, tensor, pipeline and fully sharded parallelism and explain their trade offs.
4. Reduce memory usage with mixed precision, activation checkpointing and optimiser sharding.
5. Deploy large models with high throughput inference engines using batching and caching techniques.
6. Estimate and optimise the cost of training and serving workloads.

Syllabus

Unit 21.1 Hardware fundamentals. Graphics processor architecture; streaming multiprocessors, tensor cores and memory hierarchy; arithmetic intensity and the roofline model; interconnects; tensor processing units and other accelerators.

Unit 21.2 Profiling and single device optimisation. The PyTorch profiler; identifying input pipeline stalls; kernel fusion; torch compile; custom kernels with Triton at an introductory level; efficient data loading.

Unit 21.3 Data parallelism. Distributed data parallel training; all reduce communication; gradient accumulation; large batch training and learning rate scaling; fault tolerance and checkpointing.

Unit 21.4 Model parallelism and sharding. Tensor parallelism; pipeline parallelism and scheduling; ZeRO optimiser stages; fully sharded data parallel training; DeepSpeed and Megatron concepts; combining strategies for very large models.

Unit 21.5 Memory optimisation. Mixed precision formats including bfloat16 and FP8; activation checkpointing; offloading; memory estimation for training a model of a given size.

Unit 21.6 Efficient inference. Latency and throughput metrics; key value caching; continuous batching; paged attention; speculative decoding; serving engines such as vLLM and Text Generation Inference; quantised serving; serverless and autoscaling patterns.

Unit 21.7 Cost and sustainability. Estimating floating point operations and accelerator hours; spot and reserved capacity; carbon footprint of training and inference; choosing the smallest model that meets requirements.

Practical Laboratories

- **Lab 21A: Profile and optimise.** Profile a slow training script and achieve at least a twofold speedup through targeted optimisation.
- **Lab 21B: Multi device training.** Train a transformer with distributed data parallel and fully sharded data parallel on multiple graphics processors in the cloud.
- **Lab 21C: Memory budget.** Estimate and then measure memory for fine tuning a model under different precision and checkpointing settings.
- **Lab 21D: Inference benchmark.** Serve an open weight model with vLLM and benchmark throughput and latency under varying load.

Assessment

Component	Weight	Description
Laboratory portfolio	35%	Labs 21A to 21D.

Component	Weight	Description
Scaling project	45%	Optimise the training or serving of a model of meaningful size, reporting speedups, cost savings and trade offs.
Written examination	20%	Parallelism strategies, memory estimation and inference optimisation.

Recommended Resources

- The Hugging Face Ultra Scale Playbook on training large models across many devices.
- Samyam Rajbhandari and colleagues, ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, research paper.
- Woosuk Kwon and colleagues, Efficient Memory Management for Large Language Model Serving with PagedAttention, research paper.

Competency Checkpoint

The learner is ready to progress when they can estimate, before running anything, the memory and approximate time required to fine tune a given model on given hardware, and choose a parallelism and precision strategy that makes it feasible.

Module 22: MLOps II: Production Systems at Scale

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 43 to 44	Modules 16 and 21	30 guided and 15 independent

Module Overview

Building on MLOps I, this module addresses the challenges of running many models reliably at organisational scale. Learners design feature stores, orchestrate pipelines, deploy on Kubernetes, implement comprehensive monitoring for data drift and model degradation, and automate retraining. They also study the infrastructure for operating large language model applications, often called LLMOps, including prompt management, evaluation gates and cost governance.

Learning Outcomes

By the end of this module the learner will be able to:

1. Design end to end machine learning platforms covering data, training, serving and monitoring.
2. Orchestrate reproducible training and batch inference pipelines.
3. Implement feature stores that ensure consistency between training and serving.
4. Deploy and scale model services on Kubernetes with appropriate resource management.

5. Detect and respond to data drift, concept drift and performance degradation.
6. Operate large language model applications with evaluation gates, tracing and cost controls.
7. Apply infrastructure as code and security best practices to machine learning systems.

Syllabus

Unit 22.1 Platform architecture. Components of a machine learning platform; build versus buy decisions; managed services such as Amazon SageMaker, Azure Machine Learning and Vertex AI compared with open source stacks.

Unit 22.2 Pipeline orchestration. Directed acyclic graph orchestration with Airflow, Prefect, Dagster or Kubeflow Pipelines; idempotency; backfills; parameterised and scheduled retraining.

Unit 22.3 Feature stores and data platforms. Offline and online stores; point in time correct joins; training and serving skew; streaming features with Kafka; lakehouse table formats such as Delta Lake and Apache Iceberg.

Unit 22.4 Kubernetes for machine learning. Pods, deployments and services; resource requests and limits; graphics processor scheduling; horizontal autoscaling; model serving frameworks such as KServe, BentoML and Ray Serve; Helm charts.

Unit 22.5 Monitoring and observability. Data quality monitoring; covariate, label and concept drift detection with statistical tests; performance monitoring with delayed labels; alerting design and runbooks; tools such as Evidently.

Unit 22.6 LLMOps. Prompt and configuration management; evaluation gates in deployment pipelines; tracing with OpenTelemetry based tools; guardrail services; model routing and fallback; token cost monitoring and budgets.

Unit 22.7 Infrastructure as code and security. Terraform fundamentals; secrets management; identity and access management; network isolation; supply chain security for models and dependencies; disaster recovery planning.

Practical Laboratories

- **Lab 22A: Orchestrated pipeline.** Build a scheduled pipeline that ingests data, validates it, retrains a model and registers it conditionally on evaluation.
- **Lab 22B: Kubernetes deployment.** Deploy a model service to a Kubernetes cluster with autoscaling and a canary release.
- **Lab 22C: Drift monitoring.** Simulate drift in production traffic, detect it, alert on it and trigger retraining.
- **Lab 22D: LLMOps gate.** Add an automated evaluation gate that blocks deployment of a retrieval augmented system when quality regresses.

Assessment

Component	Weight	Description
Laboratory portfolio	30%	Labs 22A to 22D.
Platform project	50%	A production style platform for a multi model use case, provisioned with infrastructure as code and fully monitored.
Incident simulation	20%	A live exercise responding to a simulated production incident, assessed on diagnosis, communication and post incident review.

Recommended Resources

- Chip Huyen, Designing Machine Learning Systems, chapters on deployment and monitoring.
- Cathy Chen and colleagues, Reliable Machine Learning, O'Reilly.
- Valliappa Lakshmanan, Sara Robinson and Michael Munn, Machine Learning Design Patterns, O'Reilly.

Competency Checkpoint

The learner is ready to progress when they can operate a monitored, automatically retrained model in a cloud environment, respond calmly to an incident, and write a blameless post incident review.

Module 23: Responsible AI, Security and Governance

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 45 to 46	Level II complete	30 guided and 15 independent

Module Overview

As AI systems influence decisions about credit, employment, healthcare, policing and public services, engineers bear real responsibility for their consequences. This module equips learners to identify, measure and mitigate harms, to secure AI systems against attack, and to comply with an evolving regulatory landscape. It treats responsibility not as an afterthought but as an engineering discipline with concrete methods, artefacts and tests.

Learning Outcomes

By the end of this module the learner will be able to:

1. Identify potential harms of AI systems across the full lifecycle using structured impact assessment.
2. Measure fairness using appropriate group and individual metrics and explain their incompatibilities.

3. Apply mitigation techniques before, during and after training.
4. Apply privacy preserving techniques including differential privacy and federated learning.
5. Assess and defend AI systems against adversarial attacks, data poisoning, model extraction and prompt injection.
6. Explain the requirements of key regulations and standards affecting AI in the United Kingdom and European Union.
7. Produce governance artefacts including model cards, datasheets and algorithmic impact assessments.

Syllabus

Unit 23.1 Ethics foundations. Ethical frameworks applied to technology; case studies of AI harms in recruitment, criminal justice, healthcare and welfare; stakeholder analysis; the role and limits of the individual engineer.

Unit 23.2 Fairness. Sources of bias in data, labels and deployment; demographic parity, equalised odds, predictive parity and calibration within groups; impossibility results; pre processing, in processing and post processing mitigations; the Equality Act 2010 and protected characteristics.

Unit 23.3 Transparency and accountability. Model cards and datasheets for datasets; explanation requirements for automated decision making; audit trails; meaningful human oversight.

Unit 23.4 Privacy. Membership inference and model inversion; differential privacy and DP SGD; federated learning; synthetic data and its limits; privacy impact assessments under UK data protection law.

Unit 23.5 Security of AI systems. The adversarial threat landscape; evasion attacks; data poisoning and backdoors; model extraction; supply chain risks from untrusted model weights; prompt injection and jailbreaks; frameworks such as MITRE ATLAS and the OWASP guidance for language model applications.

Unit 23.6 Regulation and standards. The UK principles based approach to AI regulation and the role of sector regulators; the EU AI Act and its risk based obligations; data protection law; the NIST AI Risk Management Framework; ISO/IEC 42001 AI management systems; sector requirements in finance and healthcare.

Unit 23.7 AI safety and governance in practice. Evaluation of dangerous capabilities; red teaming; responsible scaling and release practices; incident reporting; establishing organisational governance boards and review processes.

Practical Laboratories

- **Lab 23A: Fairness audit.** Audit a recidivism or lending model for group fairness, apply two mitigations and report trade offs.

- **Lab 23B: Differential privacy.** Train a model with differentially private stochastic gradient descent and chart the privacy and utility trade off.
- **Lab 23C: Adversarial attacks.** Generate adversarial examples against an image classifier and evaluate adversarial training as a defence.
- **Lab 23D: Governance pack.** Produce a model card, datasheet and algorithmic impact assessment for a previous project.

Assessment

Component	Weight	Description
Laboratory portfolio	30%	Labs 23A to 23D.
Responsible AI assessment	50%	A full responsible AI review of a realistic system, covering fairness, privacy, security and regulatory compliance, with prioritised recommendations.
Policy briefing	20%	A two page briefing for a board of directors on the regulatory obligations relevant to a proposed AI deployment.

Recommended Resources

- Solon Barocas, Moritz Hardt and Arvind Narayanan, Fairness and Machine Learning, freely available online.
- Cynthia Dwork and Aaron Roth, The Algorithmic Foundations of Differential Privacy.
- The NIST AI Risk Management Framework and the text of the EU AI Act.
- Information Commissioner's Office, Guidance on AI and Data Protection.

Competency Checkpoint

The learner is ready to progress when they can lead a responsible AI review of a proposed system, quantify its fairness and security risks, and produce documentation that would satisfy an internal audit function.

Module 24: ML System Design and Research Practice

Level	Schedule	Prerequisites	Study Hours
Level III Advanced	Weeks 47 to 48	Modules 1 to 23	30 guided and 15 independent

Module Overview

The final module integrates the entire programme. It develops the ability to design complete machine learning systems from ambiguous requirements, balancing accuracy, latency, cost, reliability and risk. It

also develops research literacy: reading papers critically, reproducing results, designing rigorous experiments and communicating findings. These capabilities are tested in the style of senior engineering interviews and design reviews, and they prepare learners equally for industry leadership or postgraduate research.

Learning Outcomes

By the end of this module the learner will be able to:

1. Translate ambiguous business requirements into precise machine learning problem formulations with measurable success criteria.
2. Design end to end architectures for common system types such as search, recommendation, fraud detection and conversational assistants.
3. Analyse trade offs among model complexity, latency, throughput, cost and maintainability.
4. Read, critique and reproduce results from machine learning research papers.
5. Design rigorous experiments, including ablations and statistical comparisons across seeds.
6. Communicate designs and findings persuasively in writing and in oral defence.

Syllabus

Unit 24.1 Problem formulation. Clarifying objectives and constraints; choosing proxy metrics that align with business outcomes; deciding whether machine learning is needed at all; defining baselines and launch criteria.

Unit 24.2 Design methodology. A repeatable framework covering requirements, data, features, models, training, serving, evaluation, monitoring and iteration; capacity estimation; designing for failure.

Unit 24.3 Canonical system designs. Search ranking; feed and content recommendation; fraud and abuse detection; advertising click prediction; visual search; enterprise knowledge assistants; real time personalisation.

Unit 24.4 Trade off analysis. Online versus batch prediction; model size versus latency; build versus buy for foundation models; precision versus recall in operational contexts; total cost of ownership.

Unit 24.5 Reading and reproducing research. How to read a paper in three passes; identifying claims, evidence and weaknesses; reproducibility challenges; using code releases responsibly.

Unit 24.6 Experimental rigour. Hypothesis driven experimentation; ablation design; multiple seeds and confidence intervals; statistical significance testing for model comparison; honest reporting including negative results.

Unit 24.7 Technical communication and leadership. Writing design documents; presenting to executives; mentoring and code review leadership; contributing to open source and publishing.

Practical Laboratories

- **Lab 24A: Design sprints.** Complete four timed system design exercises with peer and instructor feedback.
- **Lab 24B: Paper reproduction.** Reproduce a key result from a recent conference paper and report discrepancies.
- **Lab 24C: Design document.** Write a full design document for the capstone system using a professional template.
- **Lab 24D: Mock design review.** Defend a design before a panel simulating a senior engineering review.

Assessment

Component	Weight	Description
Laboratory portfolio	25%	Labs 24A to 24D.
Reproduction report	35%	A rigorous reproduction study written in the format of a workshop paper.
System design examination	40%	A ninety minute oral system design examination on an unseen problem.

Recommended Resources

- Ali Aminian and Alex Xu, Machine Learning System Design Interview, ByteByteGo.
- Martin Kleppmann, Designing Data Intensive Applications, O'Reilly.
- S. Keshav, How to Read a Paper, short article.
- Google, Rules of Machine Learning: Best Practices for ML Engineering, online guide.

Competency Checkpoint

The learner has completed Level III when they can design a complete machine learning system for an unseen problem in ninety minutes, defend their choices under questioning, and produce a reproduction study of publishable quality.

Integrated Capstone Project

Purpose of the Capstone

The integrated capstone is the culminating demonstration of competence. It runs in parallel with Level III, beginning in week thirty three and concluding with a public showcase and panel defence in week forty eight. Learners work individually or in teams of up to three, with individual contributions clearly identified. The capstone must address a genuine problem, use real or realistically complex data, and be delivered as a production style system rather than a notebook.

The capstone is assessed against the full set of programme learning outcomes. It must therefore demonstrate rigorous data work, thoughtful modelling, sound evaluation, production deployment, responsible AI analysis and professional communication. A technically impressive model with weak evaluation or no governance analysis cannot achieve a distinction.

Capstone Phases and Milestones

Weeks	Phase	Milestone and Deliverable
33 to 34	Proposal	A four page proposal defining the problem, stakeholders, data sources, success metrics, baselines, risks and an initial ethics screening. Approved by the supervisor before work proceeds.
35 to 37	Data and baseline	A validated data pipeline, an exploratory analysis report and a strong baseline model with honest evaluation.
38 to 41	Modelling	Iterative development of advanced models with tracked experiments, ablations and error analysis. A mid project review with the supervisor.
42 to 44	Engineering	Deployment as a containerised service with continuous integration, monitoring, evaluation gates and cost analysis.
45 to 46	Governance	A responsible AI assessment, model card, datasheet and security review, including a red team exercise by peers.
47 to 48	Delivery	A final technical report, design document, ten minute demonstration video, public showcase and a forty minute panel defence.

Indicative Capstone Themes

The following themes illustrate the scope and ambition expected. Learners may propose their own themes, subject to approval, and are encouraged to work with employers, charities or public bodies where data access and ethics permit.

Theme A: Regulated domain knowledge assistant. A retrieval augmented assistant that answers questions over a large corpus of regulatory or policy documents for a sector such as social care, financial services or construction safety. It must provide citations, refuse when evidence is insufficient, resist prompt injection and log interactions for audit.

Theme B: Intelligent video safety monitoring. A computer vision system that detects safety relevant events, such as missing protective equipment or unsafe zone entry, in video streams. It must operate within a defined latency budget, report precision and recall at the event level, and include a privacy impact assessment.

Theme C: Demand forecasting and replenishment. A probabilistic forecasting system for a multi location retailer or healthcare supply chain, coupled with an optimisation or reinforcement learning layer that recommends replenishment decisions, evaluated against current heuristics in simulation.

Theme D: Fraud and anomaly detection platform. A real time detection system for financial transactions or network events, combining gradient boosted models, graph features and anomaly detection, with drift monitoring, alert triage interfaces and explainability for investigators.

Theme E: Domain adapted language model. Adaptation of an open weight language model to a specialised domain such as clinical coding, legal clause classification or technical support, comparing prompting, retrieval and parameter efficient fine tuning under a rigorous evaluation framework.

Theme F: Multimodal accessibility tool. A system that combines vision, speech and language models to improve accessibility, for example describing scenes or documents for visually impaired users, with careful human evaluation involving the intended users.

Capstone Assessment Scheme

Component	Weight	Focus
Proposal	5%	Clarity of problem framing, feasibility and initial risk screening.
Technical report	30%	Data, methods, experiments, results, limitations and reproducibility.
Software and deployment	25%	Code quality, testing, deployment, monitoring and documentation.
Responsible AI pack	15%	Fairness, privacy, security and regulatory analysis with governance artefacts.
Showcase and video	5%	Clarity and impact of communication to a general audience.
Panel defence	20%	Depth of understanding and quality of justification under questioning.

Assessment Criteria and Grade Descriptors

Generic Grade Descriptors

The following descriptors apply to all summative assessments and are used by assessors to ensure consistent marking across modules. Assessors identify the band that best describes the work overall and then select a mark within that band according to the strength of the evidence.

Mark Range	Classification	Descriptor
80 to 100	Exceptional distinction	Work of outstanding quality that would be credible in a professional or research setting. Shows originality, exhaustive rigour, elegant engineering and insightful critical reflection. Limitations are anticipated and addressed with sophistication.
70 to 79	Distinction	Excellent work demonstrating deep understanding, strong technical execution and rigorous evaluation. Decisions are consistently justified with evidence. Code is clean, tested and reproducible. Communication is clear and well judged for its audience.
60 to 69	Merit	Very good work with sound understanding and competent execution. Evaluation is appropriate but may lack depth in places. Engineering is solid with minor weaknesses. Justification of decisions is mostly convincing.
50 to 59	Pass	Satisfactory work that meets the learning outcomes. Methods are applied correctly but with limited depth or critical analysis. Code works but may lack tests or documentation. Some decisions are asserted rather than justified.
40 to 49	Marginal fail	Work that partially meets the learning outcomes. There are significant gaps in understanding, flawed evaluation or substantial engineering weaknesses. Resubmission is required.
0 to 39	Fail	Work that does not meet the learning outcomes. It may be incomplete, incorrect, unreproducible or substantially derivative without acknowledgement.

Criteria for Technical Work

Technical submissions are assessed against five criteria of equal importance unless a module specification states otherwise.

Criterion	What Assessors Look For
Problem understanding	Accurate framing of the task, appropriate choice of metrics, awareness of stakeholders and constraints, and sensible baselines.
Technical correctness	Correct implementation of methods, sound mathematics, absence of leakage and bugs, and appropriate use of libraries.

Criterion	What Assessors Look For
Experimental rigour	Fair comparisons, appropriate validation design, ablations, uncertainty quantification and error analysis.
Engineering quality	Readable structure, typing, tests, documentation, reproducible environments and version control hygiene.
Critical communication	Clear reporting of results and limitations, honest discussion of risks, and writing tailored to the intended audience.

Academic Integrity and Use of AI Assistants

Learners may use AI coding assistants and language models as tools, in the same way that professional engineers do, subject to three conditions. First, every use must be declared in a short statement accompanying each submission, describing which tools were used and for what purpose. Second, the learner remains fully responsible for the correctness of every line submitted and must be able to explain it in a viva. Third, assessments explicitly designated as closed, such as timed examinations and from scratch implementation laboratories, must be completed without such assistance.

Plagiarism, collusion and fabrication of results are treated as serious misconduct. Fabricating experimental results is regarded as the most serious offence in an engineering programme because it undermines the trust on which professional practice depends.

Competency Matrix and Career Pathways

Mapping of Modules to Programme Learning Outcomes

The matrix below shows how each module contributes to the programme learning outcomes. The letter P indicates that the outcome is a primary focus of the module and is directly assessed. The letter S indicates that the outcome is supported and practised but is not the primary focus.

Module	PLO1	PLO2	PLO3	PLO4	PLO5	PLO6	PLO7	PLO8	PLO9	PLO10
1	S	P	S							S
2	P	S		S						
3	P	S		S	S					
4	P	S		S						S
5		S	P					S		
6	S	S	P							P
7		P					S			S
8	P	S	S	P						S
9	S	S		P						S
10	S	S	S	P				S		P
11	P	S			P					
12		S			P				S	S
13	S	S	S		P					
14	P	S			P	S				
15	S	S	S	P	S				S	
16		P					P		S	S
17	S	S			P	P		S		
18		S	S			P	S	P	S	
19	P	S			P			S		
20	P	S			P					
21		S			S		P		P	
22		S	S				P	S	P	
23			S					P		P
24	S								P	P

Professional Role Pathways

The programme prepares learners for a range of roles. Although every learner completes all twenty four modules, the capstone theme, elective depth within laboratories and portfolio emphasis can be aligned with a target role. The table below indicates the modules of greatest importance for each common role.

Target Role	Modules of Greatest Emphasis	Typical Responsibilities
Machine Learning Engineer	8 to 12, 16, 21, 22, 24	Building, deploying and maintaining production models and the pipelines that support them.
AI Engineer (LLM Applications)	14, 17, 18, 22, 23	Building products on foundation models, including retrieval systems, agents, evaluation and guardrails.
Data Scientist	4, 6, 8 to 10, 15	Analysing data, building predictive models and communicating insights that inform decisions.
MLOps or ML Platform Engineer	7, 16, 21, 22	Designing and running the infrastructure that allows teams to train, deploy and monitor models reliably.
Computer Vision Engineer	11, 12, 19, 21	Developing visual perception systems for inspection, safety, medical imaging and autonomy.
Applied Research Scientist	2 to 4, 14, 17, 19, 20, 24	Developing and evaluating novel methods and translating research into products.
Responsible AI Specialist	10, 23, 24	Assessing and mitigating fairness, privacy, safety and compliance risks across AI portfolios.

Indicative Competency Progression

The following descriptors summarise how capability should develop across the three levels. They are useful for self assessment, for employer appraisal frameworks and for calibrating interview expectations.

Dimension	Level I Foundations	Level II Intermediate	Level III Advanced
Mathematics	Applies core linear algebra, calculus and statistics with guidance.	Uses mathematics fluently to reason about model behaviour.	Derives and adapts methods from research literature independently.
Modelling	Implements standard algorithms and validates them correctly.	Selects and tunes appropriate models across data types.	Designs novel solutions and foundation model adaptations.

Dimension	Level I Foundations	Level II Intermediate	Level III Advanced
Engineering	Writes tested, version controlled code.	Deploys containerised services with CI and tracking.	Designs scalable platforms with monitoring and automation.
Evaluation	Uses appropriate metrics with simple validation.	Performs error analysis, calibration and interpretability.	Designs rigorous evaluation frameworks including for generative systems.
Responsibility	Understands data protection basics.	Identifies bias and explains model decisions.	Leads responsible AI reviews and regulatory compliance.
Communication	Writes clear reports for technical readers.	Presents findings to mixed audiences.	Leads design reviews and advises executives.

Building a Professional Portfolio

Employers increasingly evaluate candidates on evidence of what they have built rather than on credentials alone. By the end of the programme each learner should curate a public portfolio comprising four to six high quality projects drawn from module work and the capstone. Each project should have a clear README that states the problem, the approach, the results with honest metrics, the limitations and instructions to reproduce. Where possible, projects should include a live demonstration, a short video walkthrough and a model card.

A strong portfolio demonstrates breadth across classical machine learning, deep learning and foundation models, and depth in at least one domain. It shows engineering maturity through tests, continuous integration and deployment. Above all, it shows judgement: why a particular approach was chosen, what alternatives were rejected and what the learner would do differently with more time or data.

Delivery Guidance for Instructors

Staffing and Support

Effective delivery requires a teaching team that combines academic depth with recent industry practice. A recommended staffing ratio is one lead instructor per thirty learners, supported by one teaching assistant per fifteen learners for laboratory support and code review. Each learner should be assigned a personal mentor, ideally a practising engineer, who meets them at least fortnightly to review progress, discuss the reflective log and advise on career development.

Infrastructure Requirements

Delivery requires access to cloud computing resources with graphics processors for Modules 11 to 22, with an indicative budget of thirty to fifty accelerator hours per learner across the programme, rising for capstones involving model fine tuning. A shared Git hosting organisation, an experiment tracking server, a container registry and a small Kubernetes cluster or managed equivalent should be provisioned. All learner environments should be reproducible from version controlled configuration.

Inclusive Teaching Practice

Learners arrive with very different backgrounds. Instructors should provide multiple representations of difficult concepts, combining visual, verbal, mathematical and computational explanations. Pre recorded materials should be captioned. Mathematical content should be paired with intuition and worked numerical examples. Laboratory groups should be structured to avoid persistent domination by more experienced learners. Reasonable adjustments, such as extended time in examinations, should be available in line with institutional policy.

Keeping the Curriculum Current

The field changes rapidly, particularly in large language models, agents and inference systems. The curriculum should be formally reviewed every six months by a panel including instructors, employer representatives, recent graduates and a responsible AI specialist. Core foundations in Levels I and II are expected to remain stable, while specific tools, models and techniques in Level III, especially Modules 17, 18, 21 and 22, should be refreshed at every review. Changes should be recorded in the document control register.

Appendices

Appendix A: Forty Eight Week Delivery Schedule

The schedule below sets out the weekly focus for the full time route. Part time learners should follow the same sequence at half pace, completing each week of content over two calendar weeks.

Week	Module	Weekly Focus	Key Deliverable
1	1	Core Python, data structures and functions	Lab 1A kata
2	1	Object orientation, NumPy and Pandas	Packaged data processor
3	2	Vectors, norms, projections and matrices	Lab 2A visualisations
4	2	Linear systems, eigendecomposition, SVD and PCA	PCA from scratch
5	3	Derivatives, gradients and the chain rule	Gradient checking lab
6	3	Optimisers, convexity and autograd	Micro autograd engine
7	4	Probability, distributions and limit theorems	Simulation studio
8	4	Estimation, Bayesian inference and testing	A/B test decision memo
9	5	SQL fundamentals and advanced analytics	SQL gauntlet
10	5	Acquisition, cleaning, validation and governance	Validated pipeline
11	6	Exploration framework and visual perception	Chart critique
12	6	Advanced exploration and storytelling	EDA report and dashboard
13	7	Shell, Git and testing	Collaboration drill
14	7	Packaging, quality and continuous integration	Packaged library with CI
15	8	Linear models and regularisation	Algorithms from scratch
16	8	Classifiers, validation and pipelines	Level I integrative project
17	9	Clustering, mixtures and anomaly detection	Segmentation analysis
18	9	Bagging, boosting and stacking	Boosting benchmark
19	10	Feature engineering and selection	Encoding showdown
20	10	Metrics, calibration, tuning and explanation	Evaluation case study
21	11	Backpropagation and PyTorch fundamentals	NumPy and PyTorch networks
22	11	Training dynamics and debugging	Deep learning assignment
23	12	Convolutions and landmark architectures	CNN from scratch
24	12	Transfer learning, detection and vision transformers	Vision project
25	13	Classical NLP and embeddings	Classical baseline

Week	Module	Weekly Focus	Key Deliverable
26	13	Recurrent models, attention and evaluation	NLP assignment
27	14	Self attention and the transformer block	Attention from scratch
28	14	Training and fine tuning transformers	Mini GPT and BERT fine tuning
29	15	Statistical and machine learning forecasting	Retail demand forecasts
30	15	Recommender systems and ranking evaluation	Applied project
31	16	Tracking, versioning and serving	Prediction API
32	16	Containers, CI and observability	Level II integrative project
33	17	Pretraining, scaling and prompting	Prompt study and capstone proposal
34	17	Fine tuning, alignment, quantisation and evaluation	Adaptation project
35	18	Embeddings, ingestion and retrieval	Baseline RAG
36	18	Evaluation, agents and security	RAG or agent product
37	19	VAEs, GANs and flows	VAE and GAN labs
38	19	Diffusion models and evaluation	Generative project
39	20	Bandits, MDPs and tabular methods	Gridworld planning
40	20	Deep RL and policy gradients	RL project
41	21	Hardware, profiling and parallelism	Optimisation lab
42	21	Memory optimisation and inference serving	Scaling project
43	22	Platforms, orchestration and feature stores	Orchestrated pipeline
44	22	Kubernetes, monitoring and LLMOps	Platform project
45	23	Ethics, fairness and transparency	Fairness audit
46	23	Privacy, security and regulation	Responsible AI assessment
47	24	System design and research reproduction	Reproduction report
48	24	Design examination and capstone defence	Capstone showcase

Appendix B: Core Reading List

The following texts form the core library for the programme. Learners are not expected to read every text cover to cover; module specifications indicate the relevant chapters. Many are freely available online from their authors.

Title	Author or Source
Python for Data Analysis	Wes McKinney

Title	Author or Source
Fluent Python	Luciano Ramalho
Mathematics for Machine Learning	Deisenroth, Faisal and Ong
Introduction to Probability	Blitzstein and Hwang
Practical Statistics for Data Scientists	Bruce, Bruce and Gedeck
An Introduction to Statistical Learning with Python	James, Witten, Hastie, Tibshirani and Taylor
The Elements of Statistical Learning	Hastie, Tibshirani and Friedman
Probabilistic Machine Learning, Volumes 1 and 2	Kevin Murphy
Pattern Recognition and Machine Learning	Christopher Bishop
Deep Learning	Goodfellow, Bengio and Courville
Understanding Deep Learning	Simon Prince
Dive into Deep Learning	Zhang, Lipton, Li and Smola
Speech and Language Processing	Jurafsky and Martin
Natural Language Processing with Transformers	Tunstall, von Werra and Wolf
Build a Large Language Model from Scratch	Sebastian Raschka
AI Engineering	Chip Huyen
Designing Machine Learning Systems	Chip Huyen
Generative Deep Learning	David Foster
Reinforcement Learning: An Introduction	Sutton and Barto
Forecasting: Principles and Practice	Hyndman and Athanasopoulos
Interpretable Machine Learning	Christoph Molnar
Fairness and Machine Learning	Barocas, Hardt and Narayanan
Machine Learning Design Patterns	Lakshmanan, Robinson and Munn
Designing Data Intensive Applications	Martin Kleppmann
The Pragmatic Programmer	Thomas and Hunt

Appendix C: Glossary of Key Terms

Term	Definition
Activation function	A nonlinear function applied to the output of a neuron, such as ReLU or GELU, which allows networks to represent nonlinear relationships.
Agent	A system in which a language model plans and takes actions, typically by calling tools, in pursuit of a goal.
Attention	A mechanism that computes a weighted combination of values, where weights reflect the relevance of keys to a query.

Term	Definition
Backpropagation	The algorithm for computing gradients of a loss with respect to all network parameters by applying the chain rule in reverse.
Batch normalisation	A technique that normalises layer inputs across a mini batch to stabilise and accelerate training.
Bias and variance	The two components of reducible error: systematic error from overly simple assumptions and sensitivity to fluctuations in training data.
Calibration	The degree to which predicted probabilities match observed frequencies.
Concept drift	A change over time in the relationship between inputs and the target, which degrades model performance.
Cross validation	A resampling procedure that estimates generalisation performance by training and evaluating on multiple partitions of the data.
Data leakage	The inappropriate use of information during training that would not be available at prediction time, producing over optimistic estimates.
Diffusion model	A generative model that learns to reverse a gradual noising process in order to generate data from noise.
Embedding	A dense vector representation of an item such as a word, document, image or user, learned so that similar items are close together.
Ensemble	A combination of multiple models whose aggregated predictions are typically more accurate and robust than any individual model.
Epoch	One complete pass through the training dataset.
Feature store	A system that manages, serves and versions features consistently for training and inference.
Fine tuning	Further training of a pretrained model on a task specific dataset to adapt its behaviour.
Foundation model	A large model trained on broad data that can be adapted to a wide range of downstream tasks.
Gradient descent	An iterative optimisation algorithm that updates parameters in the direction opposite to the gradient of the loss.
Hallucination	Output from a generative model that is fluent but factually incorrect or unsupported by its sources.
Hyperparameter	A configuration value set before training, such as learning rate or tree depth, rather than learned from data.
Inference	The use of a trained model to produce predictions on new inputs.
LoRA	Low rank adaptation, a parameter efficient fine tuning method that trains small low rank matrices added to frozen weights.
Loss function	A function that quantifies the discrepancy between predictions and targets and is minimised during training.
MLOps	The set of practices for reliably developing, deploying, monitoring and maintaining machine learning systems in production.

Term	Definition
Model card	A structured document describing a model's intended use, performance, limitations and ethical considerations.
Overfitting	The condition in which a model fits noise in training data and generalises poorly to new data.
Precision and recall	Precision is the proportion of positive predictions that are correct; recall is the proportion of actual positives that are found.
Prompt injection	An attack in which malicious instructions embedded in inputs or retrieved content manipulate a language model's behaviour.
Quantisation	Representing model weights and activations with lower precision numbers to reduce memory and accelerate inference.
RAG	Retrieval augmented generation, the technique of supplying retrieved documents to a language model to ground its responses.
Regularisation	Any technique that constrains a model to reduce overfitting, such as weight penalties, dropout or early stopping.
Reinforcement learning	A paradigm in which an agent learns a policy that maximises cumulative reward through interaction with an environment.
Residual connection	A shortcut that adds a layer's input to its output, easing optimisation of very deep networks.
Scaling laws	Empirical relationships describing how model performance improves with parameters, data and compute.
Tokeniser	A component that converts raw text into discrete tokens that a language model can process.
Transfer learning	Reusing knowledge from a model trained on one task to improve learning on a related task.
Transformer	A neural architecture built on self attention and feedforward layers, dominant in language and increasingly in other modalities.
Vector database	A data store optimised for storing embeddings and performing fast approximate nearest neighbour search.

Appendix D: Preparatory Self Assessment

Prospective learners can use the following questions to judge their readiness. A learner who can answer most of them comfortably is ready to begin Module 1. A learner who finds many of them difficult should complete a short preparatory course in Python and algebra first.

1. Can you write a short program that reads a list of numbers and returns their mean and the largest value?
2. Can you solve a pair of simultaneous linear equations by hand?
3. Can you sketch the graph of a straight line and of a quadratic function and explain what the gradient of a line represents?

4. Can you calculate the probability of obtaining two heads when tossing a fair coin twice?
5. Can you explain the difference between a mean and a median and when each is more appropriate?
6. Can you install software, navigate folders and edit text files confidently on your own computer?
7. Can you read a page of technical documentation and summarise its main point in your own words?
8. Are you able to commit approximately fifteen hours each week, or eight hours each week on the part time route, for the duration of the programme?

Closing Statement

Becoming an excellent AI and machine learning engineer is a marathon rather than a sprint. This curriculum offers a demanding but coherent route, grounded in first principles and oriented firmly towards professional practice. Learners who complete it diligently will not merely know how to use today's tools; they will understand the principles that allow them to master tomorrow's tools, to build systems that work reliably in the real world, and to do so with the care and responsibility that powerful technology demands.